

Programmazione Avanzata - Giuseppe Vaglica

Studente: Giuseppe Vaglica

Ingegneria Informatica, Università di Pisa - corso di Programmazione Avanzata, A.A. 2025/2026

Disclaimer: Questi appunti sono generati a partire dal materiale didattico fornito dal professore del corso di Programmazione Avanzata. Il contenuto è stato trascritto e ripulito da Gemini 2.5 Pro, e la generazione di parti integrative da parte di AI è stata fortemente limitata ai casi essenziali in cui il professore aveva tralasciato punti chiave inerenti all'argomento trattato. L'intero contenuto è stato revisionato da me, e ho aggiunto diversi esempi e riscritto diverse parti.

Le parti generate dall'AI sono scritte in corsivo per distinguerle dal materiale originale del professore.

Ordine degli argomenti: L'ordine degli argomenti e in particolare la struttura dell'indice rispecchia (per la maggiore) l'ordine seguito dal prof a lezione e di conseguenza va dal pdf lezione0 al pdf lezione14 (dove i pdf sono gli appunti su onenote del prof).

Segnalazioni: Non posso garantire che i seguenti appunti non contengano errori, pertanto si prega di segnalare ogni errore al sottoscritto (sia errori dovuti alla conversione da markdown a pdf, sia errori grammaticali, sia errori concettuali) mediante messaggio o sui soliti contatti (@giuseppe_vaglica Whatsapp / Telegram) o mediante email a giuseppe.vaglica3@gmail.com (con oggetto "Appunti Programmazione Avanzata").

Indice

- **Programmazione Avanzata - Giuseppe Vaglica**
 - **Indice**
 - **1. Informazioni sul Corso**
 - **1.1. Docenti e Contatti**
 - **1.2. Propedeuticità e Modalità d'Esame**
 - **1.3. Testi di Riferimento**
 - **1.4. Obiettivi del Corso**
 - **1.5. Materiale Didattico e Lezioni**
 - **2. Introduzione a Java**
 - **2.1. Storia di Java**
 - **2.2. Design Goals e Versioni**
 - **2.3. JVM, JRE e JDK**
 - **2.4. Compilazione ed Esecuzione**
 - **3. Fondamentali della Programmazione in Java**
 - **3.1. Il Primo Programma: "Hello, World!"**
 - **3.2. Tipi di Dati**
 - **3.2.1. Tipi Primitivi**
 - **3.2.2. Tipi Riferimento**
 - **3.2.3. Caratteri e Codifiche Unicode**
 - **3.2.4. Tipo Stringa**
 - **3.3. Variabili e Costanti**
 - **3.4. Operazioni e Conversioni di Tipo**
 - **3.5. Strutture di Controllo**
 - **3.5.1. Enhanced for**
 - **3.6. Input/Output di Base**
 - **3.6.1. Esempio 1 (IN / OUT - somma)**
 - **3.6.2. Esempio 2 (IN / OUT - massimo)**
 - **4. Programmazione a Oggetti (OOP)**
 - **4.1. Classi e Oggetti**
 - **4.1.1. Definire una Classe**
 - **4.1.2. Costruttori**
 - **4.1.3. Metodi e Overloading**
 - **4.2. Gestione della Memoria: Stack e Heap**
 - **4.3. Passaggio dei Parametri**
 - **4.4. La keyword this**
 - **4.5. Membri static**
 - **4.5.1. Campi static**
 - **4.5.2. Metodi static**
 - **4.6. Array**
 - **4.6.1. Array Monodimensionali**
 - **4.6.2. Array Multidimensionali**
 - **4.7. Stringhe**
 - **5. Package**

- **5.1. Definizione e Scopo**
- **5.2. Dichiarazione e Nome Qualificato**
- **5.3. Convenzioni di Naming**
- **5.4. Il Package di Default**
- **5.5. Package di Sistema**
- **6. Controllo degli Accessi**
 - **6.1. Visibilità delle Classi (Top-Level)**
 - **6.2. L'Istruzione import**
 - **6.3. Gestione delle Ambiguità**
 - **6.4. Modificatori di Accesso (Membri)**
- **7. Design Pattern: Singleton**
 - **7.1. Scopo**
 - **7.2. Implementazione**
 - **7.3. Utilizzo**
- **8. Classi, File e Compilazione**
 - **8.1. Regola Generale: Classi e File**
 - **8.2. Compilazione e Dipendenze**
 - **8.3. Classi Multiple in un Singolo File**
- **9. Package e File System**
 - **9.1. Corrispondenza Package/Directory**
 - **9.2. Compilazione ed Esecuzione**
- **10. Il Classpath**
 - **10.1. Definizione**
 - **10.2. Opzioni**
- **11. Gestione della Memoria (JVM)**
 - **11.1. Heap e Garbage Collector (GC)**
 - **11.2. Garbage Collector Generazionale**
 - **11.3. Il Metodo finalize()**
- **12. Array**
 - **12.1. Informazioni di base**
 - **12.1.1. Definizione**
 - **12.1.2. Dichiarazione e Creazione**
 - **12.1.3. Accesso e Proprietà length**
 - **12.1.4. Limiti e Controllo degli Indici**
 - **12.1.5. Inizializzazione e Valori di Default**
 - **12.2. Array di Tipi Riferimento**
 - **12.3. Array Multidimensionali**
 - **12.3.1. Definizione (Array di Array)**
 - **12.3.2. Array Non Uniformi (Ragged Array)**
 - **12.4. Classi di Utilità per Array**
 - **12.4.1. System.arraycopy**
 - **12.4.2. La Classe java.util.Arrays**
- **13. Stringhe**
 - **13.1. Caratteristiche Fondamentali**
 - **13.2. Stringhe Modificabili: StringBuffer e StringBuilder**
 - **13.3. Creazione e Concatenazione**

- **13.4. Metodi Utili**
- **13.5. Confronto di Stringhe: equals() vs ==**
- **14. Il Metodo main**
 - **14.1. Argomenti da Riga di Comando**
 - **14.2. main Multipli**
- **15. Ereditarietà**
 - **15.1. Definizione e Scopo**
 - **15.2. Terminologia e Sintassi**
 - **15.3. Regole dell'Ereditarietà**
 - **15.3.1. Esempio: ContoBancario (Superclasse)**
 - **15.3.2. Esempio: ContoConInteressi (Sottoclasse)**
 - **15.4. Costruttori nelle Sottoclassi**
 - **15.4.1. La Chiamata a super()**
 - **15.5. La Classe Radice: java.lang.Object**
 - **15.5.1. Gerarchia e Metodi Ereditati**
 - **15.5.2. equals(Object o)**
 - **15.5.3. hashCode()**
 - **15.5.4. toString()**
 - **15.6. Ereditarietà e Tipi di Riferimento**
 - **15.6.1. Upcasting (da Sottoclasse a Superclasse)**
 - **15.6.2. Downcasting (da Superclasse a Sottoclasse)**
 - **15.6.3. L'Operatore instanceof**
- **16. Polimorfismo**
 - **16.1. Definizione**
 - **16.2. Regole di Invocazione (Binding)**
 - **16.3. Esempio Pratico: Gerarchia di Conti**
 - **16.4. Riepilogo: super vs this**
- **17. Classi Astratte**
 - **17.1. Definizione e Caratteristiche**
 - **17.1.1. Esempio: OggettoGrafico**
 - **17.2. Metodi Astratti**
 - **17.2.1. Esempio: disegna()**
- **18. Classi e Metodi final**
 - **18.1. Classi final**
 - **18.2. Metodi final**
- **19. Interfacce**
 - **19.1. Definizione e Scopo**
 - **19.2. Creare e Implementare un'Interfaccia**
 - **19.2.1. Esempio: Comparabile**
 - **19.3. Caratteristiche delle Interfacce**
 - **19.4. Costanti nelle Interfacce**
- **20. Tipi Enumerazione (enum)**
 - **20.1. Definizione e Uso di Base**
 - **20.2. Enum come Classi**
 - **20.2.1. Esempio Pratico**
 - **20.3. Aggiungere Stato e Metodi a un Enum**

- **20.3.1. Esempio: ColoreRGB**
- **21. Introduzione alle Eccezioni**
 - **21.1. Cos'è un'Eccezione?**
 - **21.2. Perché Usare le Eccezioni?**
 - **21.3. Gerarchia delle Eccezioni**
 - **21.3.1. La Classe Throwable**
 - **21.3.2. Eccezioni checked vs unchecked**
 - **21.4. Il Ciclo di Vita di un'Eccezione**
 - **21.4.1. Lanciare un'Eccezione (throw)**
 - **21.4.2. Catturare un'Eccezione (try-catch)**
 - **21.4.3. Ordine dei Blocchi catch**
 - **21.4.4. Il Blocco finally**
 - **21.5. Propagazione delle Eccezioni**
 - **21.5.1. Gestione Locale vs Propagazione**
 - **21.5.2. La Clausola throws**
 - **21.6. Definire Eccezioni Personalizzate**
 - **21.6.1. Creare una Classe di Eccezione**
 - **21.6.2. Esempio di Utilizzo**
 - **21.7. Buone Pratiche e Note Aggiuntive**
 - **21.7.1. Accedere ai Dettagli dell'Eccezione**
 - **21.7.2. Non Lasciare Blocchi catch Vuoti**
 - **21.7.3. Integrazione: Il Costrutto try-with-resources**
- **22. Asserzioni (Assertions)**
 - **22.1. Scopo e Utilizzo**
 - **22.2. Sintassi**
 - **22.3. Comportamento**
 - **22.4. Esempi d'Uso**
 - **22.5. Abilitare le Asserzioni**
 - **22.6. Asserzioni vs. Eccezioni**
- **23. Programmazione concorrente**
 - **23.1. Thread**
 - **23.1.1. Introduzione ai Thread**
 - **23.1.2. Creazione dei Thread**
 - **23.1.2.1. Estendendo la Classe Thread**
 - **23.1.2.2 Implementando l'Interfaccia Runnable**
 - **23.1.3. Ciclo di Vita di un Thread: Gli Stati**
 - **23.1.4. Thread e Garbage Collection**
 - **23.2. Altro sui Thread**
 - **23.2.1. Ciclo di Vita di un Thread**
 - **23.2.2. Creazione ed Esecuzione di un Thread**
 - **23.3. Proprietà e Gestione dei Thread**
 - **23.3.1. Naming dei Thread**
 - **23.1.2. Priorità e Scheduling**
 - **23.1.3. Flussi di esecuzione e oggetti**
 - **23.4. Metodi Utili della Classe Thread**
 - **23.4.1. Il Metodo sleep()**

- 23.4.2. Il Metodo `currentThread()`
- 23.5. Programmazione Concorrente e Corse Critiche
 - 23.5.1. Il Problema dell'Inconsistenza dei Dati
 - 23.5.2. Operazioni non Atomiche
- 23.6. Sincronizzazione e Mutua Esclusione
 - 23.6.1. Il Meccanismo dei Monitor in Java
 - 23.6.2. La keyword `synchronized`
 - 23.6.3. Esempio Corretto con `synchronized`
- 23.7. Il Problema dell'Interferenza
 - 23.7.1. Metodi `synchronized`
 - 23.7.2. Lock Rientranti
 - 23.7.3. Rilascio del Lock
- 23.8. Blocchi `synchronized`
 - 23.8.1. Sintassi e Funzionamento
 - 23.8.2. Equivalenza con i Metodi `synchronized`
 - 23.8.3. Quando Usare i Blocchi `synchronized`
 - 23.8.4. Sincronizzazione su Lock Multipli per Maggiore Parallelismo
- 23.9. Metodi Statici Sincronizzati
 - 23.9.1. Il Lock sulla Classe
 - 23.9.2. Esempio Pratico: Contatore Statico
- 23.10. Deadlock
 - 23.10.1. Definizione ed Esempio Teorico
 - 23.10.2. Esempio Pratico di Deadlock in Java
- 23.11. Comunicazione tra Thread: `wait()`, `notify()` e `notifyAll()`
 - 23.11.1. Oltre la Mutua Esclusione
 - 23.11.2. I Metodi Fondamentali e il "Wait-Set"
 - 23.11.3. Regole di Utilizzo e `IllegalMonitorStateException`
 - 23.11.4. Schema di Utilizzo Tipico
- 23.12. Caso di Studio: Il Problema dei Produttori e Consumatori
 - 23.12.1. Descrizione del Problema
 - 23.12.2. Implementazione della Classe Buffer
 - 23.12.3. Implementazione delle Classi Produttore e Consumatore
 - 23.12.4. Classe Principale di Avvio
- 23.13. Gestione dell'Attesa: `notify()` vs `notifyAll()`
 - 23.13.1. Il Pericolo di `notify()`
 - 23.13.2. L'Ottimizzazione `notify()`
 - 23.13.3. Esempio: Deadlock nel Produttore-Consumatore
- 23.14. Esempio Pratico: Gestore di risorse equivalenti
 - 23.14.1. Logica di Funzionamento
 - 23.14.2. Codice: Classe Gestore
 - 23.14.3. Codice: Classe Utilizzatore
 - 23.14.4. Codice: Classe Prova (Main)
- 23.15. Il Metodo `join()`
 - 23.15.1. Scopo e Funzionamento
 - 23.15.2. Versioni del Metodo
 - 23.15.3. Esempio: Analizzatore (Numero Primo)

- **23.16. Interrompere un Thread (interrupt())**
 - **23.16.1. Interruzione Cooperativa**
 - **23.16.2. Il Meccanismo interrupt()**
 - **23.16.3. Scenari di Interruzione**
 - **23.16.4. Controllare lo Stato di Interruzione**
 - **23.16.5. Esempio: Creatore ed Esecutore**
- **24. Classi Wrapper**
 - **24.1. Definizione e Scopo**
 - **24.2. Creazione di Istanze (Costruttori e valueOf)**
 - **24.3. Costanti e Metodi di Utilità**
- **25. Autoboxing e Unboxing**
 - **25.1. Autoboxing (Da Primitivo a Wrapper)**
 - **25.2. Unboxing (Da Wrapper a Primitivo)**
- **26. Java Memory Model (JMM)**
 - **26.1. Il Problema della Visibilità (Esempio)**
 - **26.2. Atomicità vs. Visibilità**
 - **26.3. La Relazione "Happens-Before"**
 - **26.4. Riordinamento delle Istruzioni (Reordering)**

1. Informazioni sul Corso

1.1. Docenti e Contatti

- **Docente:** Alessio Vecchio
- **Email:** alessio.vecchio@unipi.it
- **Telefono** 0502217463
- **Sito Web:** <http://vecchio.iet.unipi.it/vecchio>
- **Crediti:** 6 CFU
- **Altro Docente:** Prof. Vallati
- **Ricevimento:** Lunedì 8:30-10:30 (mandare prima un'email), via Teams o in presenza (Edificio A, ultimo piano, Dip. di Ingegneria dell'Informazione)

1.2. Propedeuticità e Modalità d'Esame

Propedeuticità

- Algoritmi e strutture dati
- Basi di dati

Modalità d'esame

1. **Prova pratica al calcolatore:** durata 1,5 ore, 1 esercizio. Si supera con un voto ≥ 15 .
2. **Progetto:** i dettagli sono forniti dal Prof. Vallati. Il progetto deve essere presentato il giorno della prova orale.
3. **Prova orale:** facoltativa, durata circa 20 minuti.

Novità per l'anno 25/26: prove in itinere

- **Una sola prova:** esercizio di programmazione concorrente (identico a quello della prova pratica)
 - Chi supera la prova in itinere deve discutere il progetto e può fare l'orale (facoltativo)

Altre informazioni:

- **Limite consegne:** 4 consegne ogni 7 appelli.
- **Validità scritto:** Lo scritto è valido per 3 appelli. In caso di insuccesso all'orale, lo scritto continua a essere valido.
- **Materiale consentito allo scritto:** Solo la documentazione ufficiale delle Java API (no appunti, no libri).
- **Ambiente di sviluppo:** NetBeans.

1.3. Testi di Riferimento

1. **The Java Programming Language, 4th edition (TJPL)** - K. Arnold, J. Gosling, D. Holmes
2. **Programmazione Java (HOEPLI)** - Claudio De Sio Cesari

Nota: La maggior parte degli argomenti della prima metà del corso si trova in TJPL, anche se l'ordine di presentazione e il livello di dettaglio possono differire.

Il livello di dettaglio del libro di testo (TJPL) è spesso superiore alla sintesi effettuata in classe: il libro è pensato per essere esaustivo. Di seguito sono indicate le sezioni del libro che coprono gli argomenti trattati nel corso, nell'ordine in cui appaiono nel testo, con le eventuali esclusioni specificate.

Sezioni consigliate di TJPL (The Java Programming Language) da consultare:

- Capitolo 1: tutto (escluse le sezioni 1.13 e 1.15)
- Capitolo 2: tutto (escluse le sezioni 2.5.2, 2.5.3, 2.6.3, 2.9, 2.11)
- Capitolo 3: tutto (escluse le sezioni 3.3.2, 3.3.3, 3.3.4, 3.3.5, 3.9)
- Capitolo 4: tutto (esclusa la sezione 4.3)
- Capitolo 6: tutto (esclusa la sezione 6.3.2)
- Capitolo 7: tutto (esclusa la sezione 7.5)
- Capitolo 8: dall'inizio fino alla sezione 8.4.1 (inclusa); in aggiunta consultare la sezione 8.6
- Capitolo 9: tutto (escluse le sezioni 9.1.3 e 9.6)
- Capitolo 10: tutto (escluse le sezioni 10.6 e 10.10)
- Capitolo 12: tutto (escluse le sezioni 12.2.2, 12.3.2, 12.6, 12.10.3, 12.10.4)
- Capitolo 13: solo la sezione 13.2 (escluse 13.2.7 e 13.2.8)
- Capitolo 14: tutto (escluse le sezioni 14.10.2, 14.11, 14.12, 14.13, 14.14)
- Capitolo 17: solo sezioni 17.1, 17.2, 17.3 (esclusa 17.1.3)
- Capitolo 18: tutto (escluse le sezioni 18.5 e 18.6)
- Capitolo 22: solo la sezione 22.5 (escluse 22.5.3 e 22.5.4)

Argomenti del corso non (o parzialmente) coperti da TJPL nella prima metà del corso:

- Produttori/consumatori (esempi pratici e varianti didattiche)
- Gestore di risorse equivalenti
- Meccanismi del linguaggio introdotti recentemente
- Design pattern (nuovi o trattati separatamente nel corso)

Nota: alcuni argomenti già noti (ad es. operatori, espressioni, controllo di flusso) sono trattati in TJPL (capitoli 9 e 10), ma in aula vengono spesso presentati per differenze rispetto al C++ dato che molti studenti hanno familiarità con quel linguaggio.

1.4. Obiettivi del Corso

Il corso si propone di approfondire i seguenti argomenti:

- Programmazione Object-Oriented
- Programmazione Concorrente e Distribuita
- Interfacce grafiche (Java FX)
- Sviluppo lato server (Spring)
- Interazione con Database

1.5. Materiale Didattico e Lezioni

- Il materiale didattico è disponibile su Teams.
- Le lezioni si tengono in presenza, non c'è streaming in diretta ma sono disponibili le registrazioni in differita.

2. Introduzione a Java

Java sarà il linguaggio adoperato per questo corso di programmazione avanzata. Java è un linguaggio che ha al tempo stesso un fine e uno strumento; uno strumento perché è adoperato per approfondire alcuni concetti di programmazione concorrente e vedremo alcuni framework adoperati largamente nel mondo dell'industria, mentre un fine perché continua ad essere uno dei linguaggi più adoperati.

2.1. Storia di Java

Java nasce con il nome di "progetto Oak" alla Sun Microsystem (gigante dell'informatica tra gli anni 90 e 2000 e in seguito acquisita da Oracle) nei primi anni 90, con l'idea di supportare la realizzazione di programmi per dispositivi che rientrano nel campo dell'elettronica di consumo (ricevitori del digitale terrestre, lettori di dvd, frigoriferi intelligenti, forni intelligenti, ecc.). Questi dispositivi tendono a essere eterogenei da un punto di vista hardware e hanno spesso soluzioni proprietarie anche in termini di sistemi operativi e quello che si scriveva in termini di software per questi dispositivi tendeva a non funzionare (per via dell'hardware o del sistema operativo) su dispositivi simili. Si arriva quindi all'idea di un linguaggio che permettesse portabilità nelle applicazioni, sia a livello di codice sorgente che codice binario.

Questo proposito iniziale venne abbandonato con l'esplosione del web e l'idea di applicazioni portatili a livello binario risultò vincente in tale ambito e portò alla diffusione del linguaggio. In particolare, il meccanismo che permetteva questo consiste in un client, ovvero il browser, e un server, ovvero il web server. Il web server restituiva dei documenti html, nei quali c'era un'opzione in cui venivano eseguite le cosiddette applet, ovvero piccole applicazioni in Java, che potevano essere eseguite con anche browser, hardware e sistemi operativi diversi; il tutto senza bisogno di riscrivere o ricompilare il codice.

Le applet spariscono successivamente in quanto troppo pesanti per l'hardware dell'epoca. Il calcolatore impiegava una potenza di calcolo eccessiva per eseguire le applet e in particolare la Java Virtual Machine.

Nel corso degli anni la stessa funzione si è sviluppata con Javascript diffondendosi ampiamente nell'architettura del web moderno. Nonostante la somiglianza nel nome Java e Javascript sono due cose diverse, le poche caratteristiche a comune sono le operazioni di controllo di flusso che entrambi i linguaggi ereditano dal C o dal C++. Per il resto sono linguaggi profondamente diversi (per esempio Javascript ha tipizzazione debole mentre Java ha tipizzazione forte, oppure Javascript è single-threaded e simula la concorrenza delegando le operazioni di I/O all'esterno del thread principale mantenendolo reattivo, a differenza di Java che sfrutta thread multipli).

2.2. Design Goals e Versioni

Design Goals di Java

- **Simple, object-oriented, familiar.** Semplice perché meccanismi complessi (secondo gli sviluppatori) non ci sono; per esempio, mancano i puntatori. Familiar nel senso che la maggior parte delle istruzioni di base, soprattutto quelle per il controllo del flusso sono le solite già viste in C e C++. Object-oriented perché Java si presta estremamente bene per la programmazione a oggetti e risulta più difficile scrivere del codice procedurale (sequenziale).
- **Robust, secure** (rispetto ai linguaggi del tempo). L'assenza dei puntatori, la gestione della memoria direttamente da parte del programmatore sono dei vantaggi. La robustezza deriva per esempio nel momento in cui sfioriamo i limiti di un array e generiamo delle eccezioni, in questo caso siamo obbligati a scrivere del codice per gestire questi casi.
- **Architecture neutral, portable.** Questo significa che manca l'assunzione di avere un determinato tipo di hardware o che il calcolatore abbia determinati registri. A livello di linguaggio sorgente questo si traduce in mancanza di ambiguità; gli interi sono esattamente su 32 bit in complemento a 2, stessa cosa per i long su 64 bit. La portabilità avviene non solo a livello di codice sorgente ma anche sul codice binario; questo viene fatto attraverso una rappresentazione intermedia, il cosiddetto byte-code. Il byte-code verrà poi eseguito su un calcolatore astratto, la Java Virtual Machine (JVM), implementato via software e dotato di un certo instruction set, che ha la particolarità di non avere registri, ma essere basato unicamente su uno stack (caratteristica principale che rende indipendente dall'hardware). Quello che avviene è che il codice sorgente scritto in .java viene convertito dal compilatore di Java in un byte-code che viene poi eseguito da una Java Virtual Machine (JVM) specifica per il sistema operativo.
- **High performance.** Il linguaggio come vedremo fra poco è interpretato quindi le prestazioni non sono paragonabili a quelle di codice equivalente scritto in C o C++, ma in molti casi le prestazioni sono molto vicine. In particolare, si ha questa vicinanza soprattutto quando lo stesso codice viene rieseguito più volte e si hanno prestazioni inferiori solo su codice non ancora eseguito e che quindi deve essere ancora interpretato.
- **interpreted, threaded, dynamic.** Interpretato per la presenza della Java Virtual Machine. La caratteristica del multi-thread è presente sin dalle prime versioni e serve per gestire la programmazione concorrente. I thread in Java sono il percorso di esecuzione di un programma e rappresentano la più piccola unità di esecuzione che può essere gestita in modo indipendente dallo scheduler del sistema operativo; la caratteristica di poter eseguire più thread contemporaneamente è alla base della programmazione concorrente, ovvero l'esecuzione concorrente di diverse parti del programma al fine di ottimizzare l'utilizzo della CPU, specialmente andando a sfruttare i core multipli della stessa. Infine, è dinamico perché il linking avviene contestualmente alla compilazione.

Le prime versioni di Java risalgono al 1996, ed escono con l'acronimo di JDK (Java Development Kit) e comprendono l'ambiente di base per programmare in Java con compilatore, JVM e le classi di base del linguaggio.

Nel 1998 esce J2SE, Java 2 Standard Edition: Java in versione 2 per sottolineare le profonde differenze nelle versioni e Standard Edition per separarla da Micro Edition e Enterprise Edition; il tutto è giustificato dall'impossibilità di coprire hardware profondamente diversi. ME si riferisce a semplici calcolatori, SE ai classici calcolatori e EE aggiunge funzionalità utili lato server. ME finisce nel dimenticatoio e viene rimpiazzata da sistemi come IOS e Android, che andavano di pari passo alla complessità crescente dei dispositivi.

A partire dal 2006 viene aggiornato il nome della piattaforma in Java SE e la versione in questo caso partiva da 6 (nel 2011 si ha Java SE 7 e nel 2014 Java SE 8). A seguito cambia il modello di rilascio di Java SE diventando a rilascio annuale e infatti la versione più recente è Java SE 25.

Le differenze tra Java 25 e Java 8 risultano non troppo distanti per il precedente motivo (Android ha usato per molto tempo Java 8 e solo di recente è passato a Java 11 e poi 14).

Una precisazione sulle sigle: JDK sta per Java Development Kit e JRE sta per Java Runtime Environment. JRE permette solo di eseguire le applicazioni ma non permette di compilarle e spesso è quello che gli utenti finali scaricavano. Adesso non succede più perché quello che facciamo e mettere dentro un'applicazione Java una JVM che permette l'esecuzione.

Java esiste in diverse edizioni per coprire differenti ambiti di applicazione:

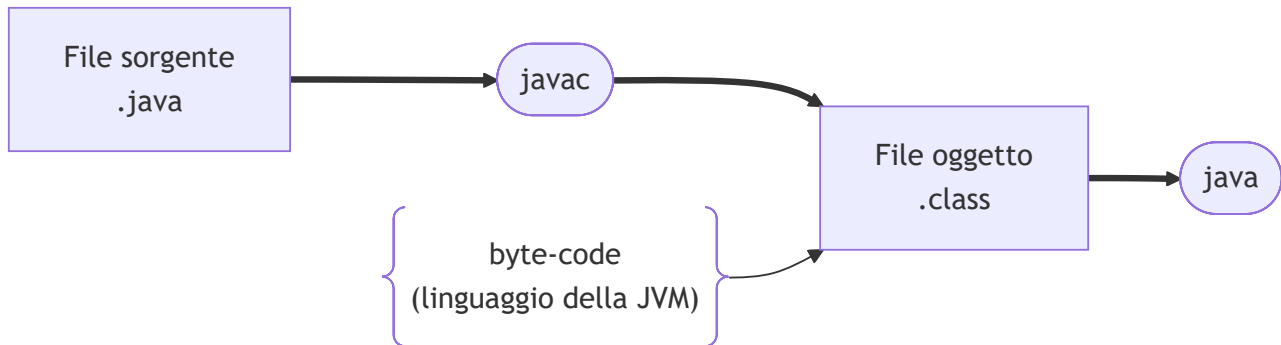
- **SE (Standard Edition):** Per applicazioni desktop e server.
- **EE (Enterprise Edition):** *Costruita sulla SE, aggiunge API per sviluppare applicazioni distribuite, multi-livello, robuste e scalabili.*
- **ME (Micro Edition):** Per dispositivi con risorse limitate come i sistemi embedded.

2.3. JVM, JRE e JDK

- **JDK (Java Development Kit):** È il kit di sviluppo, contiene tutto il necessario per scrivere e compilare programmi Java. Include il compilatore (javac) e il JRE.
- **JRE (Java Runtime Environment):** È l'ambiente di esecuzione, contiene la JVM e le librerie standard necessarie per eseguire applicazioni Java.
- **JVM (Java Virtual Machine):** È una macchina virtuale che interpreta il bytecode Java. È il componente che rende Java "architecture neutral" e "portable", astruendo dall'hardware e dal sistema operativo sottostante.

2.4. Compilazione ed Esecuzione

Il processo di sviluppo di un'applicazione Java si articola in due fasi principali:



1. **Compilazione:** Il codice sorgente (file .java) viene compilato dal comando **javac** in un formato intermedio chiamato **bytecode** (file .class).
2. **Esecuzione:** Il bytecode viene eseguito dalla JVM tramite il comando **java**. La JVM interpreta il bytecode o lo compila in codice macchina nativo "al volo" (Just-In-Time compilation, JIT) per migliorare le performance.

Su Android invece i file oggetto .class vengono eseguiti su una DVM (Dalvik Virtual Machine, basata su registri), che genera un file exe Dalvik. Successivamente, nel tempo si è aggiunto un ulteriore livello che permette tramite la compilazione di ottenere un file exe in architettura nativa ARM. Inoltre, da un paio di anni Android è diventato Kotlin-first, ovvero che considera (Google considera) Kotlin come linguaggio prioritario e raccomandato per lo sviluppo di nuove app e documentazione ufficiale; il tutto, mantenendo la compatibilità con Java (è solo un modo per Google di allontanarsi da Java e in particolare da Oracle).

Inoltre, è anche presente una versione dello JDK che è l'open JDK, completamente open source e che può essere sfruttato dalle aziende per crearne una versione personalizzata. La stessa Oracle ha il suo Oracle JDK (con il quale offre supporto ed è la sua fonte principale di guadagno), così come IBM.

3. Fondamentali della Programmazione in Java

3.1. Il Primo Programma: "Hello, World!"

In Java, tutto il codice deve essere contenuto all'interno di classi, le quali contengono delle funzioni (metodi). In questo caso abbiamo la classe Main (il nome è arbitrario) che contiene il metodo main. Il punto d'ingresso di ogni programma è il metodo main.

```
// Salviamo il codice in un file chiamato Main.java
class Main {
    // Il metodo 'main' è il punto di ingresso dell'esecuzione
    public static void main(String[] args) {
        // Stampa la stringa "Hello world!"
        System.out.println("Hello world!");
    }
}
```

Per convenzione i nomi delle classi iniziano con la lettera maiuscola (Main / NomeCompostoClasse).

Analisi del codice:

- `class Main`: Definisce una classe chiamata Main.
- `public`: Modificatore di accesso. Significa che il metodo è visibile dall'esterno della classe.
- `static`: Indica che `main` è un metodo di classe, non di istanza. Può essere chiamato senza creare un oggetto della classe Main.
- `void`: Il metodo non restituisce alcun valore.
- `String[] args`: È il parametro del metodo, un array di stringhe che contiene gli argomenti passati da riga di comando.
- `System.out.println()`: `System` è una classe, `out` è un campo statico che rappresenta lo standard output, e `println` è un metodo che stampa una stringa seguita da un "a capo".

Compilazione ed esecuzione:

```
# Per compilare
$ javac Main.java

# Per eseguire
$ java Main
```

In generale digitiamo `java` che funge da punto di partenza per l'esecuzione del nostro programma in java.

3.2. Tipi di Dati

In Java, ogni variabile ha un tipo. I tipi si dividono in due categorie: tipi primitivi e tipi riferimento.

3.2.1. Tipi Primitivi

Rappresentano valori semplici e non sono oggetti.

Tipo	Descrizione	Dimensione	Valore di Default
boolean	Valore booleano: true/false	-	false
byte	Intero con segno in C2	8 bit	0
short	Intero con segno in C2	16 bit	0
int	Intero con segno in C2	32 bit	0
long	Intero con segno in C2	64 bit	0L
float	Reale in virgola mobile, singola precisione	32 bit	0.0f
double	Reale in virgola mobile, doppia precisione	64 bit	0.0d
char	Carattere Unicode (*)	16 bit	\u0000

3.2.2. Tipi Riferimento

Puntano a oggetti (istanze di classi) memorizzati nell'heap. Il loro valore di default è null, che indica l'assenza di un oggetto.

Classi	Interfacce	Array	Enum
--------	------------	-------	------

3.2.3. Caratteri e Codifiche Unicode

Il tipo char in Java rappresenta un carattere Unicode a 16 bit.

(*) La codifica Unicode associa a ogni carattere di ogni lingua del mondo un numero detto code point. In particolare, esiste il Basic Multilingual Plane che definisce 64k caratteri. Il code point può essere codificato in tre modi:

- UTF-16: ogni code point è rappresentato su 16 bit
 - In java si usa per codificare i caratteri
 - Risulta interessante il fatto che la codifica UTF-8 coincide con la ASCII
- UTF-8: alcuni caratteri sono rappresentati su 8 bit, altri su 16 e altri ancora su 24 bit (efficiente, ma ha qualche difficoltà evidente con la gestione dell'occupazione in memoria)
 - In java si usa per codificare le stringhe
- UTF-32: ogni carattere è rappresentato su 32 bit
 - Si usa estremamente di rado questa codifica

Per dichiarare un carattere con 'a' oppure con '\u — — — — '.
codifica esadecimale
del carattere

3.2.4. Tipo Stringa

Le stringhe sono racchiuse tra doppi apici e per stringhe lunghe concatenate su più righe bisogna usare l'operatore +

```
"questa è una stringa"
```

```
"questa è una stringa" +  
"molto lunga"
```

3.3. Variabili e Costanti

La definizione di una variabile segue la sintassi `tipo nomeVariabile;`.

```
int x; // Definizione di una variabile intera  
double pi = 3.14; // Definizione e inizializzazione  
int i1, i2, i3; // Definizione di più variabili dello stesso tipo
```

Per definire una costante (una variabile il cui valore non può essere modificato dopo la prima assegnazione) si usa la parola chiave `final`.

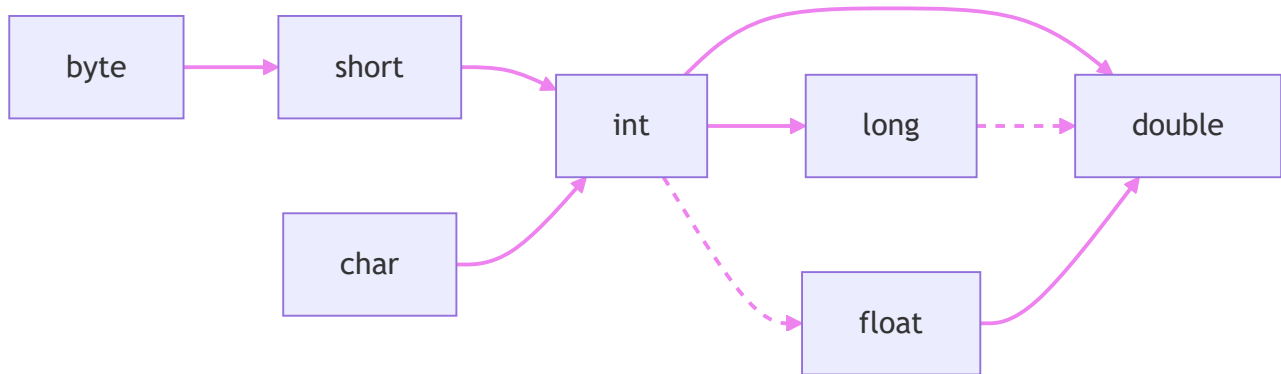
```
final int Z = 3; // Costante inizializzata  
// Z = 5; // Errore di compilazione!  
  
// Blank final: può essere inizializzata una sola volta dopo la dichiarazione  
final int W;  
W = 3; // OK  
// W = 9; // Errore!
```

3.4. Operazioni e Conversioni di Tipo

Le divisioni per zero hanno esiti diversi per interi e reali:

- **Interi:** Genera una `ArithmeticException`.
- **Reali:** Produce `Double.POSITIVE_INFINITY` o `Double.NaN` (Not a Number).

Java esegue conversioni di tipo automatiche (implicite) quando non c'è perdita di precisione. Le conversioni che potrebbero causare perdita di dati richiedono un cast esplicito.



- : conversione automatica, senza perdita di precisione
 - - -> : conversione automatica, con possibile perdita di precisione

Le conversioni sono automatiche solo seguendo il verso delle frecce e si possono fare conversioni dirette saltando gli step intermedi. Per convertire in senso opposto alla freccia si usa l'operatore cast in modo esplicito.

```
int a = 1;
long b;
b = a; // OK, conversione implicita da int a long
```

```
int c;
// c = b; // ERRORE, possibile perdita di dati
c = (int)b; // OK, cast esplicito
```

3.5. Strutture di Controllo

Le istruzioni condizionali (if-else, switch) e i cicli (for, while, do-while) sono molto simili a quelli del C/C++. La condizione deve essere sempre un'espressione booleana.

3.5.1. Enhanced for

Java introduce anche un ciclo **enhanced for** (o **foreach**), utile per iterare su tutti gli elementi di un array o di un oggetto che implementa l'interfaccia `Iterable`. Questo ha la seguente forma:

```
for(tipo variabile : set_expression)
    corpo
```

- **variabile**: assume tutti i valori identificati dalla `set_expression`
- **set_expression**: definisce l'insieme di valori da scorrere e può essere un'array oppure un oggetto che implementa l'interfaccia `java.lang.Iterable`
- **tipo**: è il tipo della variabile e deve corrispondere al tipo degli elementi da scorrere

Esempio 1 (media)

```
class Main {

    public static void main(String[] args) {
        int[] ar = new int[] {1, 2, 3}; // array di 3 interi
        double r = media(ar); // chiama il metodo media()
        System.out.println(r);
    }

    static double media(int[] v) { // metodo che restituisce double
        if(v == null || v.length == 0)
            throw new IllegalArgumentException();
        double somma = 0.0;
        for(int valore : v) {
            somma += valore;
        }
        return somma / v.length;
    }

}
```

L'esecuzione parte dal metodo `public main`, che crea un'array di 3 interi, chiama il metodo `media` passando come argomento l'array, e stampa a video la media. Nel metodo `media` gestiamo per prima cosa le eccezioni, inizializziamo `somma` e scorriamo tutti gli elementi con il `for_each` per poi calcolare la media e ritornare il `double`.

Esempio 2 (array)

```
public class Main {
    public static void main(String[] args) {
        int[] v = new int[4]; // vett. di 4 interi inizializzati a [0, 0, 0, 0]
        for(int x : v) {
            x++; // incrementa x
        }
        System.out.println(v[0]); // v rimane invariato
    }
}
```

Questo esempio serve per far capire che `x` e `v[0]` sono due valori distinti. Incrementando `x` lasceremo invariato il vettore `v`.

3.6. Input/Output di Base

A ogni programma Java sono associati tre flussi standard:

- `System.in`: Standard input (tastiera).
- `System.out`: Standard output (video).
- `System.err`: Standard error (video).

Inoltre, è sempre possibile reindirizzare il flusso tramite meccanismi messi a disposizione dal sistema operativo come `$ java Main > out.txt` per indirizzare l'output di `Main` su `out.txt`.

Per l'uscita standard usiamo i metodi `print` e `println`. Invece, per leggere dati dall'input si usa la classe `Scanner`.

```
Scanner s = new Scanner(System.in);
```

creiamo un oggetto di tipo `Scanner` che legge i dati ricevuti da `System.in`. Alcuni metodi di questa classe sono:

Tipo restituito	Nome del metodo
boolean	<code>nextBoolean()</code>
float	<code>nextFloat()</code>
double	<code>nextDouble()</code>
int	<code>nextInt()</code>
String	<code>nextLine()</code> // intera linea
String	<code>next()</code> // una sola parola
boolean	<code>hasNext()</code> // altro da leggere?

Per usare la classe Scanner, dobbiamo inserire all'inizio del file:

```
import java.util.Scanner;
```

3.6.1. Esempio 1 (IN / OUT - somma)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Primo valore? ");
        int a = sc.nextInt();
        System.out.print("Secondo valore? ");
        int b = sc.nextInt();
        int c = a + b;
        System.out.println("Risultato: " + c);
    }
}
```

3.6.2. Esempio 2 (IN / OUT - massimo)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        float m = 0; boolean primo = true;
        Scanner sc = new Scanner(System.in);
        System.out.println("Inserisci i valori (Ctrl+D per terminare)");
        while(sc.hasNext()) {
            float v = sc.nextFloat();
            if(primo) {
                m = v; primo = false;
            } else if(v > m) m = v;
        }
        if(primo) System.out.println("Nessun valore introdotto");
        else {
            System.out.println("Il massimo è: " + m);
        }
    }
}
```

4. Programmazione a Oggetti (OOP)

4.1. Classi e Oggetti

Una **classe** è un modello (blueprint) che definisce la struttura e il comportamento degli oggetti. Contiene:

- **Variabili di istanza (campi):** Definiscono lo stato degli oggetti.
- **Costruttori:** Meccanismi per la creazione e inizializzazione degli oggetti.
- **Metodi:** Codice che manipola lo stato degli oggetti.

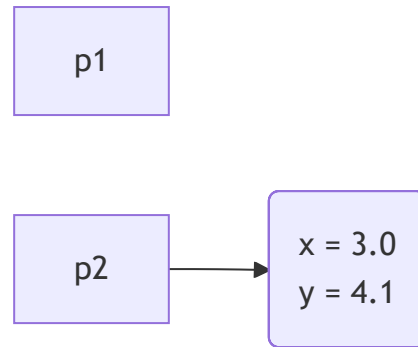
4.1.1. Definire una Classe

Di seguito un esempio di una classe Punto che rappresenta un punto in un piano.

```
class Punto {  
    // Variabili di istanza (stato dell'oggetto)  
    double x;  
    double y;  
  
    // Costruttore  
    Punto(double a, double b) {  
        x = a;  
        y = b;  
    }  
  
    // Metodi (comportamento dell'oggetto)  
    double distanza() {  
        return Math.sqrt(x * x + y * y);  
    }  
  
    void trasla(double t) {  
        x += t;  
        y += t;  
    }  
}
```

Per creare un oggetto (un'istanza di una classe) si usa la parola chiave new.

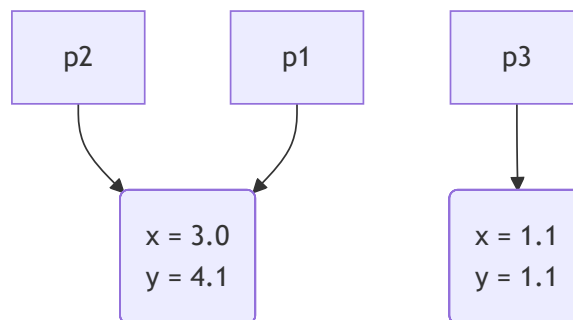
```
Punto p1; // p1 è un riferimento a oggetti di tipo Punto  
Punto p2 = new Punto(3.0, 4.1); // Crea un nuovo oggetto di tipo Punto
```



Tutti gli oggetti vengono creati con **new** in un'area di memoria detta **HEAP**. Possiamo anche fare:

```

Punto p1;
Punto p2 = new Punto(3.0, 4.1);
p1 = p2;
Punto p3 = new Punto(1.1, 1.1);
  
```

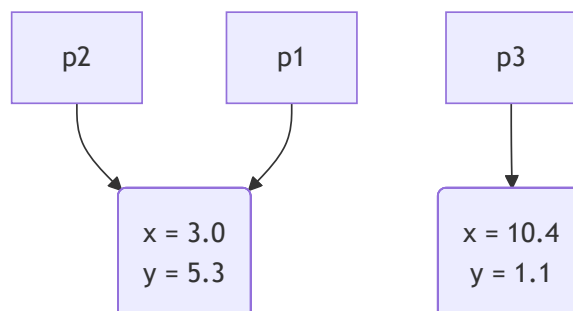


Per accedere alle variabili d'istanza e per invocare i metodi si usa la la notazione a punto (**dot notation**):

- **nomeDelRiferimento.nomeDelleVariabiliIstanza**
- **nomeDelRiferimento.nomeDelMetodo()**

```

p3.x = 10.4;
p1.y = 5.3;
double a = p2.y; // a vale 5.3
double d = p3.distanza();
p1.trasla(3.5);
  
```



4.1.2. Costruttori

Un **costruttore** è una sequenza di istruzioni usata per definire lo stato iniziale di un oggetto.

- Ha lo stesso nome della classe.
- Non restituisce alcun valore (nemmeno void).
- Viene eseguito solo insieme all'operatore new.

Se una classe non definisce alcun costruttore, Java ne fornisce uno di **default**, privo di argomenti e vuoto. Se invece si definisce almeno un costruttore, quello di default non viene più fornito automaticamente.

Una classe può avere più costruttori (overloading), a patto che si distinguano per il numero e/o il tipo degli argomenti.

```
class A {  
    double a, b;  
  
    // 1° costruttore  
    A(double c, double d) {  
        a = c;  
        b = d;  
    }  
  
    // 2° costruttore  
    A(double e) {  
        a = e;  
        b = e;  
    }  
}  
  
A a1 = new A(1.1, 2.2); // Usa il primo costruttore  
A a2 = new A(5.0); // Usa il secondo costruttore  
// A a3 = new A(); // ERRORE! IL costruttore di default non c'è più
```

4.1.3. Metodi e Overloading

Come per i costruttori, anche i metodi possono essere soggetti a **overloading**: più metodi possono avere lo stesso nome, purché si distinguano per la firma (numero e/o tipo dei parametri).

Il tipo di ritorno non è sufficiente a distinguere due metodi.

```
class Punto {  
    double x, y;  
    // ... costruttore ...  
  
    void trasla(double t) {  
        // t è detto argomento formale  
        x += t;  
        y += t;  
    }  
  
    void trasla(double tx, double ty) {  
        x += tx;  
        y += ty;  
    }  
}  
  
Punto p1 = new Punto(1.0, 2.0);  
p1.trasla(5.1); // Invoca il primo metodo trasla  
               // 5.1 è detto argomento attuale  
p1.trasla(3.4, 5.7); // Invoca il secondo metodo trasla
```

Quando invochiamo un metodo, dobbiamo fornire un valore (argomento attuale) per ogni argomento formale.

4.2. Gestione della Memoria: Stack e Heap

- **Heap:** È una zona di memoria dove vengono allocati tutti gli oggetti (creati con new). La gestione di questa memoria (deallocazione) è affidata al Garbage Collector.
- **Stack:** È una zona di memoria dove vengono memorizzati i riferimenti agli oggetti, le variabili locali e i parametri dei metodi. Ogni thread ha il proprio stack. Quando un metodo termina, il suo record di attivazione (contenente le sue variabili locali) viene rimosso dallo stack.

4.3. Passaggio dei Parametri

In Java il passaggio dei parametri è **sempre per valore**.

- **Tipi Primitivi:** Viene passata una copia del valore. Il metodo chiamato non può modificare la variabile originale del chiamante.
- **Tipi Riferimento:** Viene passata una copia del riferimento (l'indirizzo dell'oggetto). Il metodo chiamato non può modificare il riferimento originale del chiamante, ma **può modificare lo stato dell'oggetto** a cui il riferimento punta.

```
// Esempio con tipo riferimento
class Uno {
    int valore;
    Uno(int v) { valore = v; }
    void setValore(int v) { valore = v; }
}

class Due {
    void m2(Uno u) {
        // Modifica lo STATO dell'oggetto puntato da u
        u.setValore(50);
    }
}

Uno u1 = new Uno(10);
Due d1 = new Due();
d1.m2(u1);
// Ora u1.valore è 50!
```

4.4. La keyword this

La parola chiave `this` è un riferimento all'oggetto corrente (l'oggetto implicito su cui è stato invocato il metodo).

Si usa principalmente in due casi:

1. **Risolvere ambiguità:** Quando un parametro di un metodo o costruttore ha lo stesso nome di una variabile di istanza.
2. **Passare l'oggetto corrente:** Per passare un riferimento a se stesso a un altro metodo o per restituirlo.

```
class Punto {  
    double x;  
    double y;  
  
    // 'this' distingue la variabile di istanza dal parametro  
    Punto(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

// Esempio a partire dal secondo caso

```
class Numero {  
    int n;  
    Numero(int n) { this.n = n; }  
  
    Numero maggiore(Numero altro) {  
        if(n > altro.n) {  
            return this;  
        } else {  
            return altro;  
        }  
    }  
  
    void stampa() {  
        System.out.println(n);  
    }  
}
```

`this()` può essere usato anche all'interno di un costruttore per chiamare un altro costruttore della stessa classe, evitando duplicazione di codice. Deve essere la prima istruzione del costruttore.

```
class Studente {
    String nome, cognome;
    int matricola;
    double media;

    Studente(String nome, String cognome, int matricola, double media) {
        this.nome = nome;
        this.cognome = cognome;
        this.matricola = matricola;
        this.media = media;
    }

    // Chiama l'altro costruttore con un valore di default per la media
    Studente(String nome, String cognome, int matricola) {
        this(nome, cognome, matricola, 0.0);
    }
}
```

4.5. Membri static

Un membro static appartiene alla classe, non alle singole istanze. Esiste in una sola copia, condivisa da tutti gli oggetti di quella classe.

4.5.1. Campi static

I campi static sono utili per memorizzare informazioni relative a tutta la classe e non ai singoli oggetti.

```
class Veicolo {
    String proprietario;
    int id;
    // 'contatore' è condiviso tra tutti gli oggetti Veicolo
    static int contatore = 0;

    Veicolo(String p) {
        this.proprietario = p;
        this.id = contatore++;
        // Conta il numero di istanze di Veicolo
    }

    // ...
}

Veicolo v1 = new Veicolo("Mario"); // v1.id sarà 1
Veicolo v2 = new Veicolo("Luigi"); // v2.id sarà 2
System.out.println(Veicolo.contatore); // Stampa 2
// Si accede tramite il nome della classe (Veicolo.contatore)
```

Un campo static segue le stesse regole delle variabili d'istanza e può essere inizializzato allo stesso modo. Esiste solo l'eccezione che non può essere inizializzato con campi non static e non può usare metodi non static.

4.5.2. Metodi static

Sono metodi della classe e non operano su una specifica istanza.

- Non hanno un riferimento `this` in quanto non esiste il concetto di oggetto implicito.
- Possono accedere direttamente solo ad altri membri `static` (campi o metodi).
- *Vengono spesso usati come "utility methods", cioè metodi di utilità che eseguono un'operazione senza dipendere dallo stato di un oggetto (es. `Math.sqrt()`).*
- Nei metodi `static` possiamo usare oggetti, anche della stessa classe, a patto di possedere un riferimento a tali oggetti.
- Un metodo `static` può restituire oggetti di cui ha il riferimento o che esso stesso crea.

```
class Studente {  
    // ... campi nome, media, etc.  
    static int contatore;  
  
    Studente(String n) {  
        // ...  
        matricola = ++contatore;  
    }  
  
    // Metodo statico che restituisce il valore del contatore  
    public static int quantiStudenti() {  
        return contatore;  
    }  
}  
  
// Chiamata del metodo statico  
int n = Studente.quantiStudenti();
```

4.6. Array

Un **array** è un contenitore a dimensione fissa di valori dello stesso tipo.

4.6.1. Array Monodimensionali

La dichiarazione e creazione di un array sono due passi distinti.

```
int[] a; // Dichiarazione: 'a' è un riferimento a un array di interi
a = new int[5]; // Creazione: alloca un array di 5 interi nell'heap
```

- Gli indici partono da zero.
- La dimensione è accessibile tramite il campo `length` (es. `a.length` vale 5).
- L'accesso a un indice non valido (`< 0` o `>= length`) lancia una `ArrayIndexOutOfBoundsException`.

Inizializzazione:

```
// Creazione e inizializzazione esplicita
int[] v1 = {50, 100, 150};
int[] v2 = new int[]{5, 4, 3, 2, 1};

// Array di riferimenti
Studente[] s1 = new Studente[5]; // Crea un array di 5 riferimenti a Studente
// Ogni elemento di s1 è inizializzato a null
s1[0] = new Studente("Mario", "Rossi");
// Ora il primo elemento punta a un oggetto
```

4.6.2. Array Multidimensionali

In Java, un array multidimensionale è un "array di array".

```
// Crea una matrice 5x10
int[][] m1 = new int[5][10];
m1[1][2] = 33; // Assegna un valore

// Array "irregolare" (ragged array)
int[][] m2 = new int[3][]; // L'ultima dimensione non è specificata
m2[0] = new int[1];
m2[1] = new int[2];
m2[2] = new int[3];
m2[2][2] = 33;
```

4.7. Stringhe

In Java, le stringhe sono oggetti della classe `String`. Sono **immutabili**: una volta creato, il contenuto di un oggetto `String` non può essere modificato.

Per stringhe modificabili, si usano le classi `StringBuilder` (più efficiente, non thread-safe) e `StringBuffer` (thread-safe).

Creazione e Concatenazione:

```
String s1 = "Ciao";  
String s2 = "mondo";  
String s3 = s1 + " " + s2; // s3 vale "Ciao mondo"
```

Confronto tra Stringhe: `equals()` vs `==`

Questo è un punto cruciale e una comune fonte di errori.

- `==`: Confronta i **riferimenti**. Restituisce `true` solo se le due variabili puntano allo **stesso oggetto** in memoria.
- `equals()`: Confronta il **contenuto**. Restituisce `true` se le due stringhe hanno la stessa sequenza di caratteri.

Regola: Per confrontare stringhe, usare **SEMPRE** il metodo `equals()`.

```
String s1 = "abc";  
String s2 = new String("abc");  
String s3 = "abc";  
  
// s1 e s2 hanno lo stesso contenuto, ma sono oggetti diversi  
System.out.println(s1.equals(s2)); // true  
System.out.println(s1 == s2);      // false  
  
// s1 e s3 puntano allo stesso oggetto nel "String pool"  
System.out.println(s1.equals(s3)); // true  
System.out.println(s1 == s3);      // true
```

5. Package

5.1. Definizione e Scopo

- **Definizione:** Un package è un **insieme di classi tra loro correlate**.
- **Scopo:** I package ci aiutano a:
 - **Raggruppare** classi tra loro correlate.
 - **Evitare conflitti** nei nomi delle classi.
 - **Controllare** in modo fine l'accesso alle classi e ai loro membri.

5.2. Dichiarazione e Nome Qualificato

- Per inserire una classe in un package, l'istruzione package deve essere la **prima istruzione** del file sorgente.

```
// File: A.java
package miopackage;

class A {
    // ...
}
```

Inoltre, posso inserire altre classi di altri file sorgenti all'interno di miopackage:

```
// File B.java
package miopackage;

class B {
    // ...
}
```

Adesso sia A che B fanno parte del mio package.

Vediamo alcune caratteristiche dei package:

- Il **nome completo** di una classe (detto anche "nome completamente qualificato") è dato da:
`NomeDelPackage.NomeDellaClasse`.
- Esempio: `miopackage.A`.
- Questo meccanismo previene i conflitti: `miopackage.A` è diversa da `pack1.A`.
- I nomi dei package possono essere composti da più parti separate da `.`, ad esempio package `abc.def`; che porta al nome qualificato `abc.def.X`.

Per usare una classe di un altro package, si può:

1. Usare il suo nome completo (es. `java.util.Scanner sc = ...`).
2. Importarla con l'istruzione `import`, che permette di usare il nome semplice della classe.

```
import java.util.Scanner; // Importa la singola classe Scanner
import java.net.*; // Importa tutte le classi pubbliche del package java.net

public class Main {
    Scanner sc; // OK
    Socket s;    // OK
}
```

5.3. Convenzioni di Naming

- Per garantire l'unicità ed evitare conflitti, si adotta una **convenzione**: usare come parte iniziale dei nomi dei package il **proprio nome di dominio al contrario**.
- Esempi:
 - com.ibm.db. ...
 - it.unipi.email. ...
 - it.unipi.dii.db. ...

5.4. Il Package di Default

- Le classi che non vengono messe esplicitamente in un package finiscono nel **package di default** (il package senza nome).
- **Limitazione**: Le classi che sono nel package senza nome **non possono essere usate** da altro codice contenuto in package veri e propri.

- **Regola**: Mettere **sempre** le classi dentro un package.

5.5. Package di Sistema

- Tutte le librerie Java (sia di sistema che di terzi) sono strutturate in package.
- **Alcuni package di sistema**:

Package	Descrizione	Esempi di Classi
java.lang	Contiene le classi fondamentali del linguaggio	String, Object
java.io	Contiene classi dedicate all'ingresso/uscita	
java.net	Contiene classi utili ad accedere alla rete	
com.ibm.sql		
com.oracle		

6. Controllo degli Accessi

6.1. Visibilità delle Classi (Top-Level)

- Una classe può usare tutte le altre classi definite nel **suo stesso package**.
- Può usare le classi di **altri package** solo se queste sono dichiarate **public**.
- Una "top-level class" (classe non annidata) può avere solo due livelli di visibilità:
 1. **public**: La classe è visibile da tutti i package.
 2. (nessun modificatore): La classe è "non-public" (o *package-friendly*) ed è visibile solo all'interno del proprio package.
- Esempio:
 - package pack1;
 - public class A { ... }
 - class B { ... } (non-public)
 - package pack2;
 - public class C { ... }
 - class D { ... } (non-public)

Se la classe A (in pack1) prova ad usare le altre classi:

```
package pack1;
import pack2.C; // Import lecito, C è public
// import pack2.D; // Errore: D non è public

public class A {
    pack1.B b = new pack1.B(); // OK: B è nello stesso package
    pack2.C c = new pack2.C(); // OK: C è in altro package, ma è public

    // ERRORE: D è in altro package e non è public
    // pack2.D d = new pack2.D();
}
```

Questo genera errori in compilazione come: error: D is not public in pack2; cannot be accessed from outside package.

6.2. L'Istruzione import

- Per usare classi di altri package è possibile usare il **nome completamente qualificato** (es. pack2.C).
 - Per poter usare il **nome semplice** (es. C), è possibile "importare" la classe.
 - **Sintassi:** `import NomeDelPackage.NomeDellaClasse;`
 - **Posizione:** L'istruzione `import` va inserita **dopo** la dichiarazione package e **prima** della definizione della classe.
- **Import Wildcard (*)**
 - È possibile importare **tutte** le classi (pubbliche) di un package con: `import java.net.*;`
 - **Attenzione:** L'import con `*` **non** importa i package "annidati".
 - `import abc.*;` importa le classi di `abc`, ma **non** quelle di `abc.def`.
 - Per importare `abc.def` serve un import esplicito: `import abc.def.*;`
- **Import Automatico**
 - Il package `java.lang` viene **importato automaticamente** in ogni file Java.
 - Per questo motivo possiamo scrivere `String` invece di `java.lang.String`.

6.3. Gestione delle Ambiguità

- Se si importano due package (spesso con `*`) che contengono classi con lo **stesso nome**, si genera un'ambiguità.
- Esempio:
 - `import java.util.*;` (contiene `java.util.Date`)
 - `import java.sql.*;` (contiene `java.sql.Date`)
- Se si scrive `Date x = new Date();`, il compilatore non sa quale classe `Date` utilizzare.
- **Soluzione:** Bisogna usare il **nome completamente qualificato** per sciogliere l'ambiguità.
`java.util.Date x = new java.util.Date();`

Nota Integrativa: In caso di ambiguità come quella tra `java.util.Date` e `java.sql.Date`, se si importano entrambi i package tramite wildcard (`*`), il compilatore genera un errore. La soluzione è usare il nome completo (es. `java.util.Date`) per la classe desiderata. Una pratica comune è importare esplicitamente solo una delle due classi (es. `import java.util.Date;`) e usare il nome qualificato per l'altra (es. `java.sql.Date sqlDate = ...;`).

6.4. Modificatori di Accesso (Membri)

È possibile applicare modificatori di accesso ai **campi**, ai **metodi** e ai **costruttori** di una classe.

I livelli di accesso possibili sono 4:

Modificatore	Visibilità	Descrizione
public	(Massima)	Il campo o metodo può essere sempre acceduto da qualsiasi classe.
protected		Accessibile dalla stessa classe , da classi nello stesso package e dalle sottoclassi in altri package.
<nessuno>	Package-friendly	Detto anche "friendly". Accessibile dalla stessa classe e da tutte le classi dello stesso package .
private	(Minima)	Il campo o metodo può essere acceduto solo dalla stessa classe in cui è definito.

Nota: Non confondere i 4 livelli dei membri con i 2 livelli delle "top-level class" (public o *non-public*).

7. Design Pattern: Singleton

7.1. Scopo

Il Singleton è un design pattern. L'obiettivo del Singleton è fare in modo che di una classe **possa esistere un'unica istanza**.

7.2. Implementazione

Per garantire un'unica istanza, la classe `MiaClasse` viene strutturata così:

1. Campo statico privato:

- Questo campo manterrà l'unica istanza.
- È `static` perché deve esistere a prescindere dall'esistenza di oggetti.
- È `private` per non essere accessibile dall'esterno.

2. Costruttore privato:

- Questo impedisce ad altre classi di creare istanze tramite `new`.

3. Metodo statico pubblico (Factory):

- Questo metodo è l'unico punto d'accesso per ottenere l'istanza.
- La logica interna crea l'istanza solo la prima volta che viene chiamata (lazy initialization).

Codice Esempio:

```
public class MiaClasse {  
    // 1. Riferimento statico e privato all'unica istanza  
    private static MiaClasse m;  
  
    // 2. Costruttore privato: non può essere chiamato da fuori  
    private MiaClasse() {  
        // ...  
    }  
  
    // 3. Metodo statico pubblico per ottenere l'istanza  
    public static MiaClasse getIstanza() {  
        if (m == null) {  
            // Crea l'unica istanza possibile solo se non esiste già  
            m = new MiaClasse();  
        }  
        return m;  
    }  
}
```

Nota Integrativa: L'implementazione mostrata (lazy initialization) non è "thread-safe". Se due thread chiamano `getIstanza()` contemporaneamente mentre `m` è ancora `null`, entrambi potrebbero superare il controllo `if` e creare due istanze distinte, violando il pattern. Per implementazioni thread-safe, si usa la doppia verifica con `synchronized` o, più semplicemente, l'inizializzazione statica (Eager Initialization).

7.3. Utilizzo

Qualsiasi altra classe che necessita dell'istanza, la ottiene tramite `getIstanza()`:

```
// In un'altra classe:  
MiaClasse x = MiaClasse.getIstanza();  
MiaClasse y = MiaClasse.getIstanza();  
  
// x e y puntano allo stesso identico oggetto (l'unica copia) nell'Heap
```

Qualsiasi tentativo di istanziatura diretta causerà un errore di compilazione:

```
MiaClasse z = new MiaClasse(); // ERRORE: Il costruttore è privato
```

8. Classi, File e Compilazione

8.1. Regola Generale: Classi e File

- Regola generale: ogni classe è definita in un proprio file `.java`.
- Il file ha lo stesso nome della classe.
- Esempio: `public class MiaClasse { ... }` deve essere salvata nel file `MiaClasse.java`.

8.2. Compilazione e Dipendenze

Cosa succede quando `javac` compila una classe che ne usa altre?

Scenario:

- `Principale.java` contiene `public class Principale { ... }`.
- All'interno di `Principale`, usiamo `MiaClasse` (es. `MiaClasse x = new MiaClasse();`).

Processo di Compilazione:

1. Il compilatore si accorge che `Principale` usa `MiaClasse`.
2. Cerca il file compilato `MiaClasse.class`.
3. Se `MiaClasse.class` non esiste:
 - Il compilatore cerca il file sorgente `MiaClasse.java`.
 - Se lo trova, compila `MiaClasse.java` (generando `MiaClasse.class`) e poi procede con `Principale.java`.
4. Se `MiaClasse.class` esiste:
 - `javac` controlla la data di ultima modifica di `MiaClasse.class` e `MiaClasse.java`.
 - Se il file sorgente (`.java`) è più recente del file compilato (`.class`), il file `.java` viene ricompilato.

8.3. Classi Multiple in un Singolo File

- È possibile memorizzare la definizione di più classi nello stesso file .java.
- **Regola 1:** Al più una di queste classi può essere dichiarata `public`.
- **Regola 2:** Se nel file c'è una classe `public`, il nome del file .java deve obbligatoriamente coincidere con il nome della classe `public`.

Esempio:

```
// File: Tre.java

class Uno {
    // ...
}

class Due {
    // ...
}

// Solo 'Tre' è public, quindi il file DEVE chiamarsi Tre.java
public class Tre {
    // ...
}
```

- Quando si compila `$ javac Tre.java`, il compilatore produce 3 file .class: `Uno.class`, `Due.class`, e `Tre.class`.
- Svantaggio: questo approccio va usato solo quando le classi non-`public` (come `Uno` e `Due`) sono usate solo ed esclusivamente dalla classe `Tre`.

9. Package e File System

9.1. Corrispondenza Package/Directory

- Esiste una mappatura stretta tra i package e la struttura delle cartelle (directory) nel file system.
- Regola: se definisco una classe che appartiene a un package, il file sorgente (.java) deve essere posizionato in una struttura di cartelle che rispecchia il nome del package.

Esempio:

- Codice: `package abc.def; public class X { ... }`
- File System: Il file `X.java` deve trovarsi nel percorso `abc/def/`.

```
(cartella_radice)/  
└─ abc/  
    └─ def/  
        └─ X.java
```

9.2. Compilazione ed Esecuzione

- Regola: Sia per compilare (javac) che per eseguire (java), bisogna posizionarsi nella cartella radice che contiene la struttura dei package (nell'esempio, `cartella_radice`).
- Compilazione:
 - Comando: `$ javac abc/def/X.java`
 - Output: Produce il file `X.class` nella stessa cartella del sorgente (`abc/def/`).
- Esecuzione (se `X` ha un `main`):
 - Comando: `$ java abc.def.X` (si usa il nome completamente qualificato della classe, non il path del file).
 - Azione: La JVM cerca il file `X.class` partendo dalla cartella corrente e seguendo il percorso indicato dal package (`abc/def/X.class`).

10. Il Classpath

10.1. Definizione

- Il Classpath è un insieme di cartelle e/o file `.jar` (archivi) a partire dalle quali il compilatore (`javac`) e la JVM (`java`) cercano le classi (`.class`) che gli servono.
- Se il classpath non viene specificato, il valore di default è `.` (la cartella corrente).

10.2. Opzioni

- Si può specificare un classpath custom usando le opzioni `-classpath` o la sua forma abbreviata `-cp`.
- I percorsi multipli vengono separati da:
 - `:` (due punti) su sistemi Unix/Linux/macOS.
 - `;` (punto e virgola) su sistemi Windows.
- Esempio (Unix): `java -classpath /home/mario:/home/luigi abc.Z`
 - La JVM cercherà `Z.class` sia in `/home/mario/abc/` che in `/home/luigi/abc/`.
- Le classi di sistema (quelle delle librerie standard di Java) vengono trovate automaticamente, non serve specificarle nel classpath.

11. Gestione della Memoria (JVM)

11.1. Heap e Garbage Collector (GC)

- Heap: Tutti gli oggetti creati in Java (con `new`) vengono allocati in un'area di memoria chiamata Heap.
- Gestione: A differenza di altri linguaggi, in Java la deallocazione degli oggetti non è responsabilità del programmatore.
- Garbage Collector (GC): La JVM dispone di un sistema automatico per il recupero della memoria non più utilizzata, chiamato "Garbage Collector".
- Funzionamento: Un oggetto viene deallocato quando non è più "raggiungibile" (non esistono più riferimenti attivi ad esso, né dallo Stack né da altri oggetti raggiungibili).

Esempio:

```
Studente s1 = new Studente("Mario", "Rossi");  
// s1 punta all'oggetto "Mario Rossi" nell'Heap  
  
s1 = new Studente("Luigi", "Biandi");  
// s1 ora punta al nuovo oggetto "Luigi Biandi"
```

- In questo momento, l'oggetto "Mario Rossi" non è più referenziato da `s1` (né da altro). Diventa "irraggiungibile".
- Il Garbage Collector, al momento opportuno, identificherà quest'area di memoria come libera e la "recupererà".
- Il GC opera automaticamente "quando vuole lui". È possibile suggerire alla JVM di avviarlo (senza garanzia) tramite il metodo statico `System.gc()`;

11.2. Garbage Collector Generazionale

- L'Heap è spesso diviso in aree (generazioni) per ottimizzare il processo di GC.
- Young Generation (Generazione Giovane):
 - È l'area dove vengono creati i nuovi oggetti.
 - È divisa a sua volta in Eden e due aree Survivor (S0, S1).
 - Quando l'Eden si riempie, avviene una Minor GC (una pulizia "veloce" solo sulla Young Generation). Gli oggetti ancora vivi vengono spostati nelle aree Survivor.
- Old Generation (Generazione Anziana):
 - Quando un oggetto "sopravvive" a un numero sufficiente di Minor GC, viene promosso (spostato) nella Old Generation.
 - Quando la Old Generation si riempie, avviene una Major GC (o Full GC), un processo più lento che analizza l'intero Heap.
- Nota: Le note menzionano "Permgen". Quest'area, usata per i metadati delle classi, è stata sostituita in Java 8 da un'area chiamata "Metaspace", che non fa parte dell'Heap.
- Molte attività del GC sono di tipo "stop-the-world", ovvero l'intera applicazione Java viene messa in pausa mentre il GC è in esecuzione.

11.3. Il Metodo `finalize()`

- È possibile ricevere una notifica dal GC poco prima che un oggetto venga rimosso.
- Per farlo, si deve ridefinire (eseguire l'override) il metodo `protected void finalize()` ereditato da `Object`.
- Questo approccio è utile principalmente se il codice Java è collegato a risorse "native" (codice non-Java) che devono essere rilasciate manualmente.

Nota Integrativa Importante: Il metodo `finalize()` è considerato obsoleto e deprecato (fin da Java 9). Il suo utilizzo è fortemente sconsigliato. Non c'è garanzia di quando (o se) verrà chiamato. Per la gestione delle risorse (come file, connessioni di rete, o risorse native), si devono usare costrutti moderni come `try-with-resources` e l'interfaccia `AutoCloseable`.

12. Array

12.1. Informazioni di base

12.1.1. Definizione

- Un Array è un contenitore di valori dello stesso tipo.
- Gli elementi sono identificati da un indice numerico.
- Gli indici partono da 0 e arrivano a N-1, dove N è la dimensione (lunghezza) dell'array.
- La dimensione dell'array viene definita a tempo di esecuzione (quando l'array viene creato).

12.1.2. Dichiarazione e Creazione

La gestione degli array in Java avviene in due passi:

1. Dichiarazione (Riferimento):

```
int[] a;
```

- Questo crea solo un riferimento (una variabile) che potrà puntare a un array di interi. Per ora non c'è nessun array.

2. Creazione (Allocazione):

```
a = new int[5];
```

- Questo comando alloca (crea) nello Heap un nuovo array di 5 interi.
- Il riferimento a ora "punta" a questo array.
- Nota: Quando un array di tipi primitivi numerici (come `int`) viene creato, tutti i suoi elementi vengono inizializzati automaticamente al valore di default (0 per `int`).

La sintassi generale combina i due passi:

```
Tipo[] nome = new Tipo[dimensione];
```

```
// Dichiarare un riferimento 'a'
int[] a;
// 'a' punta a un nuovo array di 5 int (tutti 0)
a = new int[5];

// Dichiarare 'b' e lo fa puntare a un nuovo array di 10 int
int[] b = new int[10];

// Assegnazione: ora 'a' punta allo stesso array di 'b'
// L'array originale di 5 elementi puntato da 'a' diventa
// irraggiungibile e sarà raccolto dal GC.
a = b;
```

12.1.3. Accesso e Proprietà length

- Accesso (Lettura/Scrittura): Si usa l'operatore `[]` con l'indice desiderato.

```
int[] c = new int[3];  
c[0] = 10; // Scrive 10 nella prima posizione  
  
int k = c[0]; // Legge il valore (k ora vale 10)
```

- Proprietà `length`: Ogni array possiede una variabile membro (proprietà) `length` che ne indica la lunghezza (dimensione).

```
int[] a = new int[5];  
int l = a.length; // l vale 5  
  
float[] c1 = new float[23];  
l = c1.length; // l vale 23
```

12.1.4. Limiti e Controllo degli Indici

- Un riferimento ad array può essere riassegnato per puntare a un nuovo array (anche di dimensione diversa).

```
float[] a1 = new float[5];  
int l = a1.length; // l vale 5  
a1 = new float[10];  
l = a1.length; // l vale 10
```

- In Java non esiste l'aritmetica dei puntatori. L'operazione `a1++` è un errore.
- L'operatore di selezione `[]` controlla che non vengano sforati i limiti dell'array.
- L'indice deve essere sempre: $0 \leq \text{indice} < \text{dimensione}$.
- Se si accede a un indice non valido (es. `v1[7]` su un array di 5 elementi), il sistema lancia un'eccezione a tempo di esecuzione: `ArrayIndexOutOfBoundsException`.

12.1.5. Inizializzazione e Valori di Default

- È comune iterare su un array usando la sua proprietà `length`.

```
int[] v = new int[10];
for (int i = 0; i < v.length; i++) v[i] = i;
```

- Quando un array viene creato, i suoi elementi vengono inizializzati automaticamente ai loro valori di default:

■ 0 per i tipi numerici (`int`, `double`, `float...`), `false` per i boolean e `null` per i tipi riferimento (oggetti, stringhe, altri array).

- È possibile fornire i valori iniziali (inizializzatore) al momento della creazione:

```
int[] v3 = {50, 100, 150};    // Sintassi 1 (implicita)
int[] v4 = new int[] {5, 4, 3, 2, 1};    // Sintassi 2 (esplicita)
```

12.2. Array di Tipi Riferimento

Consideriamo una classe `Studente`.

- Quando si crea un array di un tipo riferimento (un oggetto), si crea un array di riferimenti.

```
Studente[] s1 = new Studente[5];
```

- Questo non crea 5 oggetti `Studente`. Crea un array di 5 elementi, e tutti gli elementi sono riferimenti a `Studente`, inizializzati a `null`.
- Se si tenta di usare un elemento prima di averlo istanziato, si ottiene un errore, ad esempio `double m = s1[0].getMedia();` // ERRORE!. Questo errore genera un'eccezione `NullPointerException`, perché si sta cercando di chiamare un metodo su un riferimento `null`.
- È necessario istanziare manualmente ogni oggetto nell'array:

```
s1[0] = new Studente("Mario", "Rossi");
// Ora s1[0] punta a un oggetto Studente valido
```

- È possibile usare un inizializzatore anche per gli array di oggetti:

```
Studente[] s2 = {
    new Studente("Luigi", "Bianchi"),
    new Studente("Sara", "Gialli")
};
```

12.3. Array Multidimensionali

12.3.1. Definizione (Array di Array)

- In Java, un array multidimensionale è in realtà un array di array.
- `int[][] m1 = new int[5][10];`
- Questa sintassi crea:
 1. Un array "principale" di 5 elementi.
 2. Ogni elemento di questo array è un riferimento a un array di `int[10]`.
 3. Ciascuno dei 5 array `int[10]` viene creato automaticamente.
- Si accede agli elementi con la doppia parentesi: `m1[1][2] = 33;`
- Per iterare, si usano cicli annidati:

```
for (int i = 0; i < m1.length; i++) {  
    // m1.length è 5 (righe)  
    for (int j = 0; j < m1[i].length; j++) {  
        // m1[i].length è 10 (colonne)  
        m1[i][j] = ...;  
    }  
}
```

12.3.2. Array Non Uniformi (Ragged Array)

- È possibile omettere l'ultima dimensione (o le ultime) durante la creazione. `int[][] m2 = new int[3][];`
- Questo crea un array di 3 elementi (`m2`), dove ogni elemento è un riferimento di tipo `int[]`, inizializzato a `null`.
- Qualsiasi tentativo di accesso `m2[0][0] = 33;` genera un `NullPointerException`.
- Le "righe" (gli array interni) devono essere create manualmente. Questo permette di avere dimensioni diverse per ogni riga (array "ragged" o non uniformi).

```
// Creo un array non uniforme (triangolare)  
for (int i = 0; i < m2.length; i++) {  
    m2[i] = new int[i + 1];  
}  
m2[2][2] = 33; // Ora è valido
```

12.4. Classi di Utilità per Array

12.4.1. System.arraycopy

- La classe System offre un metodo statico arraycopy per copiare efficientemente elementi da un array a un altro.
- Sintassi: `public static void arraycopy(Object src, int srcPos, Object dest, int destPos, int length)`
- Copia length elementi dall'array src (partendo da srcPos) all'array dest (partendo da destPos).

12.4.2. La Classe java.util.Arrays

- Questa classe (java.util.Arrays) mette a disposizione molti metodi statici utili per la manipolazione di array (ordinamento, ricerca, ecc.).
- Esempi:
 - `public static void sort(double[] a)`: Ordina l'array specificato in ordine numerico ascendente.
 - `public static int binarySearch(int[] a, int key)`: Cerca un valore (key) in un array di int usando l'algoritmo di ricerca binaria.
 - Importante: L'array deve essere ordinato prima di chiamare binarySearch, altrimenti i risultati sono indefiniti.

13. Stringhe

13.1. Caratteristiche Fondamentali

- Le stringhe in Java sono istanze della classe `String`.
- Non sono array di caratteri terminati da `\0` (come in C/C++).
- Le stringhe sono immutabili — una volta creato un oggetto `String`, il suo contenuto non può più essere modificato.

13.2. Stringhe Modificabili: `StringBuffer` e `StringBuilder`

- Per manipolare stringhe "modificabili" esistono due classi apposite:
 - `StringBuffer`: È thread-safe, ovvero può essere manipolato da più thread contemporaneamente.
 - `StringBuilder`: Non è thread-safe, ma è generalmente più veloce; usarlo da un singolo thread.

13.3. Creazione e Concatenazione

- Le stringhe si creano comunemente usando i letterali (`"`). `String s1 = "ABC";`
- L'operatore `+` viene usato per concatenare le stringhe.

```
String s2 = "Ciao";  
String s3 = "mondo";  
String s4 = s2 + " " + s3; // s4 vale "Ciao mondo"
```

- L'operatore `+` può essere applicato anche tra stringhe e valori di altro tipo (es. `int`). Il valore non-stringa viene convertito automaticamente in stringa.

```
int v = 4;  
String r = "Rocky";  
String t = r + v; // t vale "Rocky4"
```

```
int v = 4, w = 5;  
String r = "Rocky";  
// Operazione 1: Stringa + int -> Stringa. Poi Stringa + int -> Stringa  
String y = r + v + w; // ("Rocky" + 4) + 5 -> "Rocky4" + 5 -> "Rocky45"
```

```
// Operazione 2: int + int -> int. Poi int + Stringa -> Stringa  
String y2 = v + w + r; // (4 + 5) + "Rocky" -> 9 + "Rocky" -> "9Rocky"
```

13.4. Metodi Utili

- `public int length()`: Restituisce la lunghezza della stringa.
 - Nota: È un metodo (`length()`), a differenza della proprietà `length` degli array.
 - `String sc = "ABC"; int k = sc.length(); // k vale 3`
- `public char charAt(int i)`: Restituisce il carattere alla posizione `i`.
 - `char c = sc.charAt(1); // c vale 'B'`
- `public String substring(int beginIndex, int endIndex)`: Restituisce la sotto-stringa.
 - Nota: L'indice `beginIndex` è incluso, `endIndex` è escluso.
 - `String se = "Smiles"; String sb = se.substring(1, 5); // sb vale "mile"`
- Da `char[]` a `String`:

```
char[] ar = {'c', 'i', 'a', 'o'};
String sd = new String(ar); // sd vale "ciao"
```

13.5. Confronto di Stringhe: `equals()` vs `==`

Questo è un punto fondamentale e fonte comune di errori.

Regola: Per confrontare le stringhe usare sempre il metodo `equals()`. Non usare l'operatore `==`.

- `equals()` (confronto di contenuto): restituisce `true` se le stringhe hanno gli stessi caratteri nello stesso ordine.
- `==` (confronto di riferimenti): confronta i riferimenti (gli indirizzi di memoria) e restituisce `true` solo se i due riferimenti puntano allo stesso oggetto nello heap.

```
String s1 = "abc";
String s3 = "ab";
char x = 'c';
s3 = s3 + x; // s3 ora vale "abc"
```

```
// s1 e s3 hanno lo stesso contenuto? SÌ.
boolean b1 = s1.equals(s3); // b1 vale true
```

```
// s1 e s3 puntano allo stesso oggetto? NON È GARANTITO.
// (In questo caso, 's3' è un nuovo oggetto creato dalla concatenazione)
boolean b2 = (s1 == s3); // b2 è probabilmente false
```

Anche se due letterali identici (es. `String s1 = "abc";` e `String s4 = "abc";`) potrebbero puntare allo stesso oggetto per ottimizzazione (interning), non si deve mai fare affidamento su questo comportamento. Usare sempre `equals()`.

14. Il Metodo main

14.1. Argomenti da Riga di Comando

- La signature del metodo main è `public static void main(String[] args)`.
- Il parametro args è un array di String che riceve i valori forniti sulla riga di comando al momento dell'esecuzione.

```
// Classe Stampa.java
public class Stampa {
    public static void main(String[] args) {
        for (int i = 0; i < args.length; i++) {
            System.out.println(args[i]);
        }
    }
}
```

- Esecuzione: `java Stampa mario luigi peach`
- Risultato: L'array args conterrà: `args[0] = "mario", args[1] = "luigi", args[2] = "peach"`.

14.2. main Multipli

- Più classi all'interno dello stesso progetto possono avere un metodo main.
- Motivi:
 1. Test (main di prova): una classe (es. `Studente`) può avere un main al suo interno usato solo per testare la classe stessa.
 2. Punti di ingresso multipli: l'applicazione può funzionare in modalità diverse. Ad esempio, un main per l'interfaccia grafica (GUI) e uno per l'interfaccia a riga di comando (CLI).
- Quando si esegue il comando `java`, si specifica quale main (ovvero, quale classe) si intende avviare (es. `java GUI` o `java CLI`).

15. Ereditarietà

15.1. Definizione e Scopo

- L'ereditarietà è un meccanismo che ci permette di definire nuove classi a partire da classi già esistenti.
- Le nuove classi sono specializzazioni delle classi esistenti.
- Scopo: aiuta a riusare il codice. Si esprimono solo le differenze rispetto alla classe base.

15.2. Terminologia e Sintassi

- Sintassi: si usa la parola chiave `extends`.

```
class A { ... } // Superclasse  
class B extends A { ... } // Sottoclasse
```

- Terminologia:
 - Classe A (quella da cui si eredita): Superclasse, classe base, classe padre, supertipo.
 - Classe B (quella che eredita): Sottoclasse, classe derivata, classe figlia, sottotipo.
- La relazione che si esprime è del tipo "è un".
 - Un Triangolo è un Poligono.
 - Un Quadrato è un Rettangolo.
 - Viceversa non è vero: un Poligono non è necessariamente un Triangolo.

15.3. Regole dell'Ereditarietà

- In Java, l'ereditarietà è singola: ogni classe può avere una sola superclasse.
- Una classe può avere molte sottoclassi.
- Quando `B extends A`:
 - B eredita i campi e i metodi (non privati) di A.
 - B può aggiungere nuovi campi e nuovi metodi.
 - B può ridefinire (override) il comportamento dei metodi ereditati da A.
- Accesso private:
 - Una sottoclasse (B) non può accedere ai membri private della superclasse (A), anche se li eredita.

15.3.1. Esempio: ContoBancario (Superclasse)

```
public class ContoBancario {  
    // Campi  
    private double bilancio;  
    private final String intestatario;  
    private final int numero;  
    // Campo statico per generare numeri di conto univoci  
    private static int prossimo = 1;  
  
    // Costruttore  
    public ContoBancario(String i) {  
        intestatario = i;  
        numero = prossimo++;  
        // bilancio è inizializzato a 0.0 di default  
    }  
  
    // Metodi  
    public double getBilancio() { return bilancio; }  
  
    public String getIntestatario() { return intestatario; }  
  
    public int getNumero() { return numero; }  
  
    public void deposita(double d) { bilancio += d; }  
  
    public void preleva(double d) { bilancio -= d; }  
  
    public void trasferisci(ContoBancario c, double d) {  
        // Usa i suoi stessi metodi  
        preleva(d);  
        c.deposita(d);  
    }  
  
    public String toString() {  
        return "intestatario: " + intestatario +  
            ", numero: " + numero +  
            ", bilancio: " + bilancio;  
    }  
}
```

Nota integrativa sui campi: l'uso di *final* per *intestatario* e *numero* significa che, una volta impostati (tipicamente nel costruttore), il loro valore non può più essere cambiato.

15.3.2. Esempio: ContoConInteressi (Sottoclasse)

Un ContoConInteressi è un ContoBancario, ma con l'aggiunta di un tasso di interesse e un'operazione per aggiungere gli interessi maturati.

```
public class ContoConInteressi extends ContoBancario {  
    // 1. Aggiunge un nuovo campo (la differenza)  
    private final double tasso;  
  
    // 2. Costruttore  
    public ContoConInteressi(String i, double t) {  
        // Chiama il costruttore della superclasse (ContoBancario)  
        // Deve essere la prima istruzione  
        super(i);  
        tasso = t;  
    }  
  
    // 3. Aggiunge un nuovo metodo  
    public double getTasso() {  
        return tasso;  
    }  
  
    public void aggiungiInteressi() {  
        // Usa metodi ereditati (getBilancio e deposita)  
        double q = getBilancio() * tasso;  
        deposita(q);  
    }  
  
    // 4. Ridefinisce (Override) un metodo esistente  
    public String toString() {  
        // Richiama il toString() di ContoBancario  
        String s = super.toString();  
        // Aggiunge la nuova informazione  
        return s + ", tasso: " + tasso;  
    }  
}
```

15.4. Costruttori nelle Sottoclassi

15.4.1. La Chiamata a `super()`

- La sottoclasse deve chiamare un costruttore della superclasse per inizializzare i campi ereditati da quest'ultima.
- Per chiamare il costruttore della superclasse, si usa la parola chiave `super`.
- La chiamata `super(...)` deve essere la prima istruzione del costruttore della sottoclasse.
- Caso 1 (Opzionale): Se la superclasse è dotata di un costruttore privo di argomenti (no-arg constructor), la chiamata `super()` può essere omessa e verrà inserita implicitamente dal compilatore.
- Caso 2 (Obbligatorio): Se la superclasse è dotata soltanto di costruttori con argomenti, è obbligatorio chiamarne uno esplicitamente tramite `super(...)`.

15.5. La Classe Radice: `java.lang.Object`

15.5.1. Gerarchia e Metodi Ereditati

- Esiste una classe di sistema chiamata `Object` (package `java.lang`).
- Se una classe non dichiara esplicitamente di estenderne un'altra, essa è implicitamente figlia (sottoclasse) di `Object`.
- Tutte le classi Java, quindi, ereditano i metodi di `Object`.

La gerarchia reale dell'esempio precedente è:

```
Object
^
|
ContoBancario
^
|
+--- ContoConInteressi
+--- ContoGratuito
```

- I metodi di `Object` si dividono in due categorie:
 1. **Sincronizzazione tra thread:** `wait()`, `notify()`, `notifyAll()`.
 2. **Utilità generale:** `equals()`, `hashCode()`, `toString()`.

15.5.2. equals(Object o)

- Scopo: `public boolean equals(Object o)` confronta l'oggetto implicito (`this`) con l'oggetto `o`. Restituisce `true` se sono "uguali", `false` altrimenti.
- Implementazione di `Object`: L'implementazione predefinita della classe `Object` corrisponde all'operatore `==` (confronto dei riferimenti).
 - Esempio: `A a1 = new A(); A a2 = new A(); -> a1.equals(a2)` restituisce `false`, perché `a1` e `a2` sono due oggetti distinti.
- Override (Ridefinizione): Ogni classe può ridefinire (fare l'override) del metodo `equals` secondo il proprio criterio (es. `String` lo ridefinisce per controllare i caratteri).

Come ridefinire `equals` (Esempio `Studente`): si stabilisce che due studenti sono "uguali" se hanno la stessa matricola.

```
public boolean equals(Object o) {  
    // 1. Controllo se 'o' è nullo  
    // 2. Controllo se 'o' è dello stesso tipo (classe)  
    if (o == null || getClass() != o.getClass()) {  
        return false;  
    }  
    // 3. Cast: converto 'o' al tipo Studente  
    Studente s = (Studente) o;  
  
    // 4. Confronto Logico (basato sui campi)  
    return matricola == s.matricola;  
}
```

15.5.3. hashCode()

- Scopo: `public int hashCode()` restituisce un numero intero (hash code) per l'oggetto su cui viene applicato.
- Implementazione di `Object`: L'implementazione predefinita restituisce valori diversi per oggetti diversi (spesso l'indirizzo di memoria interno alla JVM, a partire dal quale l'oggetto è allocato).

Contratto `equals/hashCode`:

Regola Fondamentale: Molte classi contenitore (es. `HashMap`) richiedono che se `a.equals(b)` restituisce `true`, allora anche `a.hashCode()` e `b.hashCode()` devono restituire lo stesso valore.

Override (Esempio `Studente`): Poiché `equals` si basa sulla `matricola`, anche `hashCode` deve basarsi su di essa.

```
@Override
public int hashCode() {
    return matricola;
}
```

15.5.4. toString()

- Scopo: `public String toString()` restituisce una rappresentazione in formato stringa dell'oggetto.
- Implementazione di `Object`: L'implementazione predefinita restituisce la stringa `NomeDellaClasse@HashCodeDellOggetto` (es. `Studente@A39X..`).
- Chiamata Automatica: `toString()` viene richiamato automaticamente quando si concatena un oggetto a una stringa.
 - `String x = "Studente: " + s1;` è come scrivere `String x = "Studente: " + s1.toString();`.

Override (Esempio `Studente`):

```
@Override
public String toString() {
    return nome + ", " + cognome;
    // L'output di 'x' diventa: "Studente: Mario, Rossi"
}
```

15.6. Ereditarietà e Tipi di Riferimento

15.6.1. Upcasting (da Sottoclasse a Superclasse)

- Un riferimento di tipo superclasse può puntare a un oggetto di tipo sottoclasse.
- Questa conversione (upcasting) è implicita e sempre sicura.
- Un ContoConInteressi è un ContoBancario.

```
// Oggetto reale: ContoConInteressi
ContoConInteressi cci = new ContoConInteressi("Mario", 0.01);

// Riferimento di tipo Superclasse: ContoBancario
ContoBancario cb = cci; // OK (Upcasting)

// Riferimento di tipo Object (Superclasse di ContoBancario)
Object o = cci; // OK (Upcasting)
```

15.6.2. Downcasting (da Superclasse a Sottoclasse)

- È possibile convertire un riferimento da supertipo a sottotipo, ma la conversione deve essere esplicita usando un cast (Tipo).
- La JVM esegue un controllo a tempo di esecuzione.
- Se l'oggetto reale non è compatibile con il tipo richiesto, viene lanciata un'eccezione `ClassCastException`.
- Regola: Quello che conta è il tipo reale dell'oggetto, non il tipo del riferimento.

```
ContoConInteressi cci = new ContoConInteressi("Peach", 0.04);
ContoBancario y = cci; // Upcasting (y punta a un ContoConInteressi)

// Esempio 1: Downcasting SICURO
// y punta a un ContoConInteressi, che è compatibile.
ContoConInteressi z = (ContoConInteressi) y; // OK

// ---

ContoBancario a = new ContoBancario("Toad");
// (a punta a un ContoBancario)

// Esempio 2: Downcasting NON SICURO
// L'oggetto reale è ContoBancario, non ContoConInteressi.
ContoConInteressi b = (ContoConInteressi) a; // ERRORE: ClassCastException
```

15.6.3. L'Operatore instanceof

- Per evitare `ClassCastException`, si può controllare il tipo reale dell'oggetto prima di eseguire il cast usando `instanceof`.
- L'operatore restituisce `true` se il riferimento `r` punta a un oggetto compatibile con `ContoConInteressi`.

```
if (y instanceof ContoConInteressi) {  
    // Sicuro: 'y' è un ContoConInteressi  
    ContoConInteressi z = (ContoConInteressi) y;  
    // ...  
}
```

16. Polimorfismo

16.1. Definizione

- Polimorfismo: (Dal greco "molte forme"). Un'entità del programma (come un riferimento) può assumere diverse forme (puntare a tipi di oggetti diversi).
- Si basa sul fatto che un riferimento di tipo superclasse può puntare a istanze delle sue sottoclassi.
- Quando un metodo viene invocato su tale riferimento, l'esecuzione specifica che avviene dipende dalla ridefinizione fatta dalla sottoclasse a cui l'oggetto appartiene.

16.2. Regole di Invocazione (Binding)

Ci sono due regole chiave che governano il polimorfismo:

1. Tipo del Riferimento (Compile-time): Il tipo del riferimento determina l'insieme dei metodi che possono essere invocati. Il compilatore controlla se il metodo esiste in quel tipo.
2. Tipo dell'Oggetto (Runtime): Il tipo reale dell'oggetto (nello heap) determina quale implementazione del metodo viene effettivamente eseguita.

Esempio: Tre riferimenti (cci, cb, o) puntano tutti allo stesso oggetto ContoConInteressi.

```
ContoConInteressi cci = new ContoConInteressi("Luigi", 0.02);  
ContoBancario cb = cci;  
Object o = cci;
```

Analisi delle Chiamate:

- cci.aggiungiInteressi(); -> OK. Il riferimento cci è ContoConInteressi, il metodo esiste.
- cb.aggiungiInteressi(); -> ERRORE (Compilazione). Il riferimento cb è ContoBancario, che non ha il metodo aggiungiInteressi.
- o.aggiungiInteressi(); -> ERRORE (Compilazione). Il riferimento o è Object, che non ha il metodo aggiungiInteressi.
- cb.deposita(100.0); -> OK. Il metodo deposita esiste in ContoBancario.
- o.toString(); -> OK. Il metodo toString esiste in Object.
 - Polimorfismo: Viene eseguito il toString() di ContoConInteressi (la classe dell'oggetto reale).

16.3. Esempio Pratico: Gerarchia di Conti

Il polimorfismo permette di scrivere codice generico che funziona per un'intera gerarchia.

```
public class ContoGratuito extends ContoBancario {
    private static final double COSTO_OPERAZIONE = 1.0;
    // costo operazioni aggiuntive
    private final int operazioniGratuite;
    private int operazioniEseguite;

    public ContoGratuito(String i, int n) {
        super(i);
        operazioniGratuite = n;
    }

    // nuovo metodo
    public int operazioniGratuiteRimanenti() {
        return operazioniGratuite - operazioniEseguite;
    }

    public void deposita(double d) {
        operazioniEseguite++;
        super.deposita(d);
        // richiama il deposita() di ContoBancario
    }

    public void preleva(double d) {
        operazioniEseguite++;
        super.preleva(d);
        // richiama il preleva() di ContoBancario
    }

    public void assegnaCosti() {
        if(operazioniEseguite > operazioniGratuite) {
            double importo = (operazioniEseguite -
                operazioniGratuite)*COSTO_OPERAZIONE;
            super.preleva(importo);
            // richiamo preleva() della classe base in modo
            // che non conti come operazione
        }
        operazioniEseguite = 0;
    }

    public String toString() {
        String s = super.toString();
```

```
        // richiamo toString() di ContoBancario
        return s + ", op. gratuite: " + operazioniGratuiteRimanenti();
    }
}
```

```
ContoBancario x = new ContoBancario("Mario");
ContoGratuito y = new ContoGratuito("Luigi", 100);
```

```
// Chiamo trasferisci(y, ...). 'y' è un ContoGratuito.
x.trasferisci(y, 200.0);
```

Il codice scritto per ContoBancario funziona automaticamente e correttamente anche per ContoGratuito.

16.4. Riepilogo: super vs this

- Entrambe sono parole chiave usate nel contesto dell'ereditarietà, ma con scopi diversi.

this

- È un riferimento all'oggetto implicito (l'istanza corrente).
- `this()`: Se usato come prima istruzione in un costruttore, chiama un altro costruttore della stessa classe.
- `this.metodo()`: Usato per disambiguare (o semplicemente per accedere) ai membri dell'istanza corrente.

super

- Non è un riferimento.
- `super()`: Se usato come prima istruzione in un costruttore, chiama il costruttore della superclasse.
- `super.metodo()`: Permette di invocare un metodo della superclasse che è stato ridefinito (overridden) nella classe corrente.

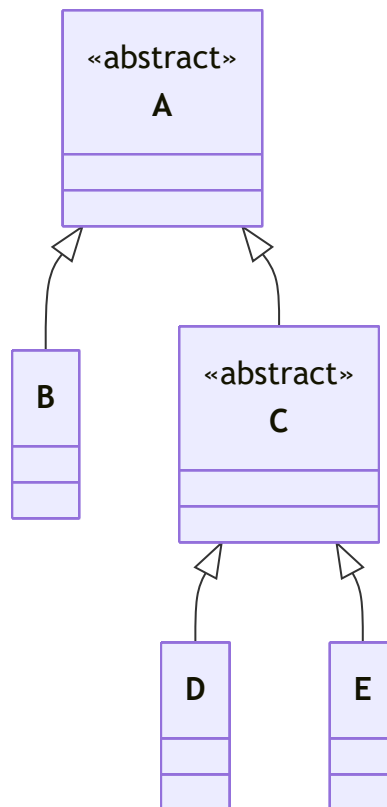
17. Classi Astratte

17.1. Definizione e Caratteristiche

Una **classe astratta** non può essere istanziata. Viene usata come classe base nella definizione di nuove classi. È utile quindi come "modello" per la definizione di nuove classi.

Non è possibile creare oggetti (istanze) di una classe astratta, ma è possibile creare riferimenti del suo tipo.

Consideriamo la seguente gerarchia di classi:



In questo scenario:

- **Non posso creare istanze** di A e C perché sono astratte.
- **Posso creare istanze** di B, D, ed E.
- **Posso creare riferimenti** di tipo A, B, C, D, ed E.

17.1.1. Esempio: OggettoGrafico

Immaginiamo di voler modellare oggetti grafici generici. Non tutti gli oggetti grafici sono uguali, ma tutti condividono delle proprietà, come la posizione (x, y) e il comportamento di potersi spostare.

```
// "Modello" per tutti gli oggetti grafici
abstract class OggettoGrafico {
    int x, y;

    void sposta(int nuovaX, int nuovaY) {
        x = nuovaX;
        y = nuovaY;
    }
}

// Classi concrete che estendono il modello
class Bottone extends OggettoGrafico {
    // ... implementazione specifica del Bottone
}

class CasellaDiTesto extends OggettoGrafico {
    // ... implementazione specifica della CasellaDiTesto
}
```

Il seguente codice mostra cosa è permesso e cosa non lo è:

```
// ERRORE: non posso istanziare una classe astratta
OggettoGrafico og1 = new OggettoGrafico();

// OK: posso creare un riferimento di tipo astratto
// che punta a un'istanza di una sottoclasse concreta
OggettoGrafico og2 = new Bottone();

// OK: il metodo è ereditato dalla classe base astratta
og2.sposta(100, 200);
```

17.2. Metodi Astratti

In una classe astratta, alcuni metodi possono essere etichettati come `abstract`.

Un **metodo astratto** è un metodo senza implementazione (corpo). Obbliga le sottoclassi non astratte a fornire una loro specifica implementazione.

Regola fondamentale: Se una classe contiene anche un solo metodo `abstract`, allora l'intera classe deve essere dichiarata `abstract`.

17.2.1. Esempio: `disegna()`

Tutti gli oggetti grafici devono poter essere disegnati, ma il modo in cui un Bottone viene disegnato è diverso da una CasellaDiTesto.

```
abstract class OggettoGrafico {
    int x, y;

    void sposta(int nuovoX, int nuovoY) {
        x = nuovoX;
        y = nuovoY;
    }

    // Metodo astratto: non ha corpo, solo la firma.
    // Obbliga le sottoclassi a implementarlo.
    abstract void disegna();
}

class Bottone extends OggettoGrafico {
    // Definizione obbligatoria del metodo astratto
    void disegna() {
        // ... codice per disegnare un bottone ...
    }
}

class CasellaDiTesto extends OggettoGrafico {
    String testo;

    // Definizione obbligatoria del metodo astratto
    void disegna() {
        // ... codice per disegnare una casella di testo ...
    }

    void altroMetodo() {
        // ...
    }
}
```

```
}  
}
```

Grazie al **polimorfismo**, possiamo trattare oggetti diversi in modo uniforme. Quando invochiamo il metodo `disegna()` su un riferimento di tipo `OggettoGrafico`, Java eseguirà dinamicamente la versione corretta del metodo in base al tipo reale dell'oggetto.

```
OggettoGrafico og1 = new Bottone();  
OggettoGrafico og2 = new CasellaDiTesto();  
  
og1.sposta(100, 200);  
og2.sposta(300, 400);  
  
// Viene eseguito il disegna() della classe Bottone  
og1.disegna();  
  
// Viene eseguito il disegna() della classe CasellaDiTesto  
og2.disegna();
```

Nota del professore: È possibile, all'interno di un metodo concreto di una classe astratta, chiamare i metodi astratti della stessa classe. Sarà compito della sottoclasse concreta fornire l'implementazione che verrà eseguita a runtime.

```
abstract class X {  
    void m1() {  
        // ...  
        m2(); // Chiamata a un metodo astratto  
        // ...  
    }  
  
    abstract void m2();  
}
```

18. Classi e Metodi final

La parola chiave `final` impone delle restrizioni sull'ereditarietà e sulla modifica.

18.1. Classi final

Una classe dichiarata `final` **non può essere estesa** (cioè, non può avere sottoclassi).

```
final class Y {  
    // ...  
}  
  
// ERRORE DI COMPILAZIONE: non posso estendere una classe final  
class Z extends Y {  
    // ...  
}
```

Le classi `final` sono spesso usate per motivi di sicurezza o per garantire l'immutabilità di una classe, come nel caso della classe `String` in Java.

18.2. Metodi final

Un metodo dichiarato `final` **non può essere sovrascritto** (overridden) nelle sottoclassi.

Questo è utile per garantire che il comportamento di un metodo non venga alterato da una sottoclasse.

```
class C {  
    final void mioMetodo() {  
        // ... implementazione che non deve essere cambiata ...  
    }  
}  
  
class D extends C {    // OK  
    // ERRORE DI COMPILAZIONE: non posso sovrascrivere un metodo final  
    void mioMetodo() {  
        // ...  
    }  
}
```

Nota importante: Quando una classe è `final`, tutti i suoi metodi sono implicitamente `final`.

Un caso d'uso tipico è per metodi critici per la sicurezza.

```
class Autenticazione {  
    final boolean controllaPassword(String p) {  
        // ... Logica complessa e sicura ...  
    }  
}
```

Rendendo il metodo `final`, si impedisce che una sottoclasse possa ridefinirlo in modo da restituire sempre `true`, bypassando i controlli di sicurezza.

19. Interfacce

19.1. Definizione e Scopo

In Java, un'**interfaccia** è una raccolta di metodi abstract. Definisce un protocollo di comportamento a cui le classi possono decidere di aderire.

*A differenza di una classe astratta, che può fornire sia implementazioni complete che metodi astratti (definendo un "è un" rapporto parziale), un'interfaccia definisce un "contratto" puramente astratto. Specifica **cosa** una classe può fare, ma non **come** lo fa. Una classe che implementa un'interfaccia promette di fornire un'implementazione per tutti i metodi definiti in quel contratto.*

19.2. Creare e Implementare un'Interfaccia

Si definisce un'interfaccia usando la parola chiave `interface`.

19.2.1. Esempio: Comparabile

Definiamo un'interfaccia che stabilisce un protocollo per confrontare oggetti.

```
// I metodi in un'interfaccia sono implicitamente public e abstract.
public interface Comparabile {
    int compara(Object o);
}
```

Una classe che implementa un'interfaccia deve fornire l'implementazione di tutti i suoi metodi.

```
public class Data implements Comparabile {
    int giorno, mese, anno;

    public Data(int g, int m, int a) {
        giorno = g;
        mese = m;
        anno = a;
    }

    // Implementazione obbligatoria del metodo dell'interfaccia
    public int compara(Object o) {
        Data d = (Data) o; // Cast dell'oggetto
        if (anno != d.anno) return anno - d.anno;
        if (mese != d.mese) return mese - d.mese;
        return giorno - d.giorno;
    }
}
```

Anche una classe completamente diversa come ContoBancario può aderire allo stesso protocollo.

```
public class ContoBancario implements Comparabile {  
    double bilancio;  
    // ...  
  
    public int compara(Object o) {  
        ContoBancario c = (ContoBancario) o;  
        if (bilancio < c.bilancio) return -1;  
        if (bilancio > c.bilancio) return 1;  
        return 0;  
    }  
}
```

19.3. Caratteristiche delle Interfacce

1. **Riferimenti di tipo Interfaccia:** È possibile creare riferimenti di tipo interfaccia, ma non istanze. Un riferimento di tipo interfaccia può puntare a un'istanza di qualsiasi classe che implementa quell'interfaccia.

```
Data d1 = new Data(10, 5, 1950);
Data d2 = new Data(11, 6, 2000);
ContoBancario cb1 = new ContoBancario(/*...*/);

// OK: c1 punta a un oggetto Data, che implementa Comparabile
Comparabile c1 = d1;

// OK: posso chiamare solo i metodi dell'interfaccia
int res = c1.compara(d2);

// ERRORE a runtime (ClassCastException): cb1 e d1 non sono compatibili
// anche se entrambi implementano Comparabile
cb1.compara(d1);
```

2. **Implementazione Multipla:** Una classe può implementare tutte le interfacce che vuole, separandole con una virgola.

```
class X implements Interf1, Interf2, Interf3 {
    // Deve fornire l'implementazione di TUTTI i metodi
    // di Interf1, Interf2 e Interf3.
}
```

3. **Estensione e Implementazione:** Una classe può estendere un'altra classe e, al tempo stesso, implementare una o più interfacce. La clausola `extends` deve sempre precedere `implements`.

```
class A extends B implements I {
    // ...
}
```

19.4. Costanti nelle Interfacce

Un'interfaccia può contenere anche delle costanti.

Le costanti definite in un'interfaccia sono implicitamente **public**, **static** e **final**.

```
public interface Verbose {  
    int SILENT = 0;  
    int NORMAL = 1;  
    int VERBOSE = 2;  
  
    void setVerbosity(int level);  
    int getVerbosity();  
}
```

Una classe che implementa l'interfaccia "eredita" le costanti e le può usare direttamente.

```
public class Logger implements Verbose {  
    private int level;  
    // ... implementazione dei metodi ...  
  
    public void log(String message) {  
        if (level == SILENT) { // Uso diretto della costante  
            return;  
        }  
        // ...  
    }  
}
```

Le classi che non implementano l'interfaccia possono comunque usare le sue costanti usando la notazione `NomeInterfaccia.NOME_COSTANTE`.

```
int logLevel = Verbose.SILENT;
```

20. Tipi Enumerazione (enum)

Un **enum** in Java è un tipo speciale i cui valori possibili sono un insieme fisso di costanti, note al momento della dichiarazione. Assomigliano ai tipi enumerazione del C/C++.

20.1. Definizione e Uso di Base

Si definisce un enum con la parola chiave enum.

```
public class Main {  
  
    // Definizione del tipo enum Colore con i suoi possibili valori  
    enum Colore { VERDE, GIALLO, ROSSO };  
  
    public static void main(String[] args) {  
        // Dichiarazione di una variabile di tipo Colore  
        Colore c1 = Colore.VERDE;  
  
        // ERRORE: bisogna sempre usare il nome del tipo enum  
        // Colore c2 = GIALLO;  
  
        // Gli enum sono ottimi per gli switch  
        switch (c1) {  
            case VERDE: // Qui non serve il prefisso "Colore."  
                System.out.println("Vai");  
                break;  
            case GIALLO:  
                System.out.println("Rallenta");  
                break;  
            case ROSSO:  
                System.out.println("Fermati");  
                break;  
        }  
    }  
}
```

20.2. Enum come Classi

In Java, gli enum sono molto più potenti che in C/C++. In effetti, sono delle **classi speciali** che estendono implicitamente la classe `java.lang.Enum`.

Da questa classe ereditano alcuni metodi utili:

- `String toString()`: Restituisce il nome dell'enumeratore come stringa (es. "GIALLO").
- `int ordinal()`: Restituisce la posizione ordinale della costante (a partire da 0).
- `public static E[] values()`: Restituisce un array contenente tutte le costanti dell'enum, nell'ordine in cui sono state dichiarate.
- `static E valueOf(String n)`: Restituisce la costante dell'enum con il nome specificato. Lancia `IllegalArgumentException` se non c'è corrispondenza.

20.2.1. Esempio Pratico

```
import java.util.Scanner;

class Main {
    enum Colore { VERDE, GIALLO, ROSSO };

    public static void main(String[] args) {
        Colore c1 = Colore.VERDE;
        System.out.println(c1); // Stampa VERDE (chiama toString())

        // Itera su tutti i valori dell'enum
        for (Colore c2 : Colore.values()) {
            System.out.println(c2 + " " + c2.ordinal());
        }
        // Output:
        // VERDE 0
        // GIALLO 1
        // ROSSO 2

        Scanner sc = new Scanner(System.in);
        System.out.print("Inserisci un colore: ");
        String parola = sc.next(); // es. "GIALLO"

        // Converte una stringa in un valore enum
        Colore c3 = Colore.valueOf(parola);
        System.out.println("Hai inserito: " + c3);
    }
}
```

Essendo delle classi, è possibile **ridefinire il comportamento dei metodi**, come `toString()`, per avere un output più leggibile.

```
enum Colore2 {  
    VERDE, GIALLO, ROSSO;  
  
    // Ridefinisco toString()  
    public String toString() {  
        switch (this) {  
            case VERDE: return "Verde";  
            case GIALLO: return "Giallo";  
            case ROSSO: return "Rosso";  
            default: return null;  
        }  
    }  
}  
  
// System.out.println(Colore2.ROSSO);  
// -> Stampa "Rosso" invece di "ROSSO"
```

20.3. Aggiungere Stato e Metodi a un Enum

Essendo delle classi, è possibile aggiungere ai tipi enum **stato** (attributi), **costruttori** e **metodi** personalizzati.

20.3.1. Esempio: ColoreRGB

Creiamo un enum che non solo rappresenta i colori, ma memorizza anche le loro componenti RGB e può calcolarne la luminosità.

```
enum ColoreRGB {  
    // Le costanti invocano il costruttore  
    VERDE(0, 1, 0),  
    GIALLO(1, 1, 0),  
    ROSSO(1, 0, 0);  
  
    // Stato: attributi privati e final  
    private final double[] componenti;  
  
    // Costruttore (è implicitamente private)  
    ColoreRGB(double r, double g, double b) {  
        componenti = new double[]{r, g, b};  
    }  
  
    // Metodo personalizzato  
    public double luminosita() {  
        return 0.2126 * componenti[0]  
            + 0.7152 * componenti[1]  
            + 0.0722 * componenti[2];  
    }  
  
    private String comp() {  
        return "(R=" + componenti[0]  
            + ", G=" + componenti[1]  
            + ", B=" + componenti[2] + ")";  
    }  
  
    public String toString() {  
        String s = null;  
        switch (this) {  
            case VERDE: s = "Verde"; break;  
            case GIALLO: s = "Giallo"; break;  
            case ROSSO: s = "Rosso"; break;  
        }  
        return s + " " + comp();  
    }  
}
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        for (ColoreRGB c : ColoreRGB.values()) {  
            System.out.println(c + " lum: " + c.luminosita());  
        }  
    }  
}
```

L'output del programma sarà:

```
Verde (R=0.0, G=1.0, B=0.0) lum: 0.7152  
Giallo (R=1.0, G=1.0, B=0.0) lum: 0.9278  
Rosso (R=1.0, G=0.0, B=0.0) lum: 0.2126
```

21. Introduzione alle Eccezioni

21.1. Cos'è un'Eccezione?

Un'eccezione è un meccanismo utile alla **rappresentazione di situazioni di errore** e alla loro gestione.

Alcuni esempi di situazioni di errore includono:

1. Tentare di scrivere su una porzione del file system senza i permessi necessari.
2. Una disconnessione di rete durante la comunicazione con un server esterno.
3. Accedere a un array usando un indice non valido (es. negativo o fuori dai limiti).
4. Chiamare un metodo su un riferimento null.
5. Un errore interno della JVM, come un fallimento nel linking.

Nelle situazioni 1 e 2 ha senso prevedere azioni di recupero (es. chiedere nuovamente i permessi, ritentare la connessione). Nelle situazioni 3, 4 e 5, invece, non ha senso prevedere azioni di recupero: si tratta di errori di programmazione o problemi gravi che richiedono la correzione del codice sorgente o l'intervento dell'utente.

In Java, queste situazioni di errore sono rappresentate come **oggetti eccezione**, ovvero istanze di opportune classi.

21.2. Perché Usare le Eccezioni?

Senza eccezioni, il codice per la gestione degli errori è spesso mescolato con il codice "normale" dell'applicazione, rendendolo meno leggibile e più difficile da mantenere.

Approccio tradizionale (senza eccezioni):

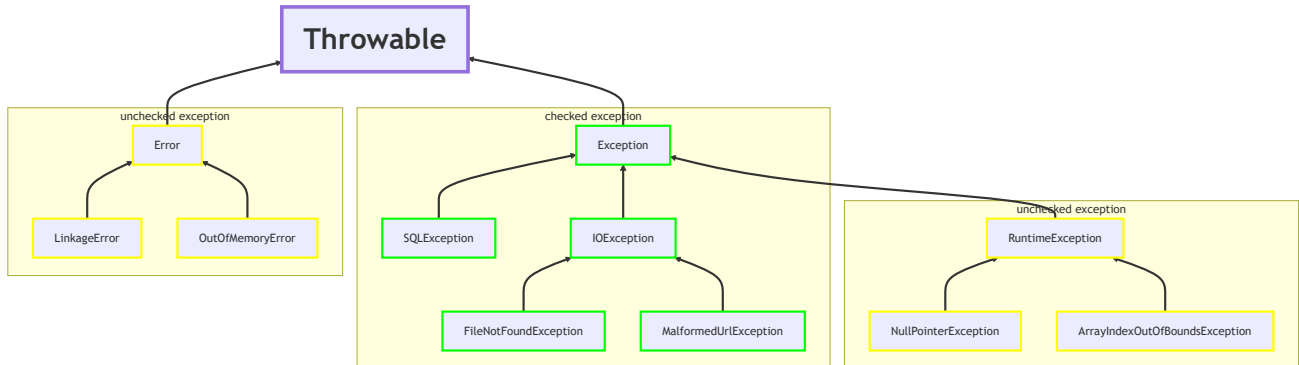
```
if (funzione1() == CODICE_ERRORRE) {  
    // Codice per la gestione dell'errore  
}  
  
// Codice "normale"  
  
if (funzione2() != OK) {  
    // Altro codice per la gestione dell'errore  
}
```

Il vantaggio principale del meccanismo delle eccezioni è che permette di **mantenere separato il codice "normale" da quello per la gestione degli errori**.

21.3. Gerarchia delle Eccezioni

21.3.1. La Classe Throwable

Tutte le eccezioni in Java sono organizzate in una gerarchia di classi che ha come radice la classe **Throwable**.



21.3.2. Eccezioni checked vs unchecked

Le eccezioni si dividono in due categorie principali:

1. checked (controllate)

Per le eccezioni checked, il compilatore **obbliga** il programmatore a fornire codice di gestione. *Tutte le sottoclassi di Exception che non sono anche sottoclassi di RuntimeException sono checked.*

2. unchecked (non controllate)

Non c'è obbligo di gestione per le eccezioni unchecked. *Tutte le sottoclassi di Error e RuntimeException sono unchecked.*

21.4. Il Ciclo di Vita di un'Eccezione

Ci sono due fasi fondamentali nella vita di un'eccezione: il **lancio** e la **cattura**.

21.4.1. Lanciare un'Eccezione (throw)

Si lancia un'eccezione quando si rileva una situazione di errore, utilizzando l'istruzione **throw**.

```

if (condizioneDiErrore) {
    throw new Exception("Messaggio descrittivo dell'errore");
}
  
```

Nota del professore: Generalmente si crea e lancia l'oggetto eccezione in un'unica istruzione (throw new Exception();), poiché il riferimento all'oggetto appena creato non serve ad altro.

21.4.2. Catturare un'Eccezione (try-catch)

Per catturare e gestire un'eccezione si usa il costrutto **try-catch**.

Le istruzioni che possono lanciare un'eccezione vengono inserite nel blocco `try`. Se un'eccezione viene lanciata, l'esecuzione del blocco `try` si interrompe e il controllo passa al primo blocco `catch` corrispondente.

Sintassi:

```
try {  
    // Istruzioni che possono lanciare eccezioni  
    // Queste istruzioni non vengono eseguite se un'eccezione  
    // viene lanciata prima di esse.  
} catch (TipoEccezione1 e1) {  
    // Codice di gestione per TipoEccezione1  
} catch (TipoEccezione2 e2) {  
    // Codice di gestione per TipoEccezione2  
}
```

Un blocco `catch` è "corrispondente" se il tipo dell'eccezione specificato nel `catch` è lo stesso tipo dell'eccezione lanciata, o un suo **supertipo**. Ad esempio, un `catch (Exception e)` può catturare una `IOException`.

21.4.3. Ordine dei Blocchi catch

Quando i tipi di eccezioni sono in relazione di ereditarietà, l'ordine dei blocchi catch è fondamentale.

Bisogna mettere **prima i blocchi catch relativi ai tipi più specifici** (sottoclassi) e poi quelli più generici (superclassi).

Esempio CORRETTO:

```
try {  
    // ...  
} catch (IOException ioe) { // Sottoclasse specifica  
    // Gestione di IOException  
} catch (Exception e) {    // Superclasse generica  
    // Gestione di altre eccezioni  
}
```

Esempio ERRATO (errore di compilazione):

```
try {  
    // ...  
} catch (Exception e) {    // Questo "acchiappa" anche le IOException  
    // ...  
} catch (IOException ioe) { // Questo blocco non verrebbe mai eseguito!  
    // ...  
}
```

21.4.4. Il Blocco finally

Il costrutto try-catch può includere anche un blocco **finally**.

Il codice contenuto nel blocco finally **viene eseguito sempre**:

- Se il blocco try termina normalmente.
- Se un'eccezione viene lanciata e catturata da un catch.
- Se un'eccezione viene lanciata ma non catturata (il `finally` viene eseguito prima che l'eccezione venga propagata al chiamante).

Il blocco finally è spesso usato per contenere **codice di pulizia** (es. chiusura di file, connessioni di rete, etc.), garantendo che le risorse vengano rilasciate correttamente.

Esempio di pulizia di risorse:

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        Writer w = null;
        try {
            w = new FileWriter("out.txt");
            for (int i = 0; i < 10; i++) {
                w.write(String.valueOf(i)
                    + System.getProperty("line.separator"));
            }
        } catch (IOException ioe) {
            System.out.println("Errore: " + ioe.getMessage());
        } finally {
            try {
                if (w != null) {
                    w.close();
                }
            } catch (IOException e) {
                // EAT: In questo caso specifico, sopprimere l'eccezione
                // sulla close è accettabile, perché l'errore originale
                // (nel blocco try) è più importante da segnalare.
            }
        }
    }
}
```

21.5. Propagazione delle Eccezioni

21.5.1. Gestione Locale vs Propagazione

Quando un metodo contiene codice che può lanciare un'eccezione checked, abbiamo due possibilità:

1. **Gestirla localmente:** Racchiudere il codice in un blocco try-catch.
2. **Propagarla al chiamante:** Dichiarare che il metodo può lanciare quell'eccezione.

21.5.2. La Clausola throws

Per propagare un'eccezione checked, la si deve dichiarare nella firma del metodo usando la clausola **throws**.

```
public class Pila {  
    public Object pop() throws PilaVuotaException {  
        if (empty()) {  
            throw new PilaVuotaException();  
        }  
        // Il resto del codice non viene eseguito se l'eccezione è lanciata  
        // ...  
    }  
}
```

Quando un'eccezione viene lanciata e non gestita localmente, essa si propaga "a ritroso" lungo la **catena di chiamate dei metodi** (call stack), finché non viene trovato un blocco catch in grado di gestirla. Se nessun gestore viene trovato, il thread di esecuzione termina e il programma si arresta.

Un metodo può dichiarare di lanciare più tipi di eccezioni, separandoli con una virgola.

```
void mioMetodo() throws IOException, SQLException {  
    // ...  
}
```

21.6. Definire Eccezioni Personalizzate

21.6.1. Creare una Classe di Eccezione

È possibile definire nuovi tipi di eccezioni creando classi che estendono `Exception` (per eccezioni checked) o `RuntimeException` (per unchecked).

Esempio:

// Gerarchia: Exception -> PortaRottaException -> SerraturaRottaException

```
public class PortaRottaException extends Exception {
    public PortaRottaException() {
        super();
    }

    public PortaRottaException(String message) {
        super(message);
    }
}

public class SerraturaRottaException extends PortaRottaException {
    public SerraturaRottaException() {
        super();
    }

    public SerraturaRottaException(String message) {
        super(message);
    }
}
```

[Integrazione: è buona norma fornire sempre almeno due costruttori: uno senza argomenti e uno che accetta una stringa (il messaggio di errore). Questi costruttori tipicamente invocano i costruttori corrispondenti della superclasse (super()).]

21.6.2. Esempio di Utilizzo

```
if (portaDifettosa) {
    throw new PortaRottaException("La porta principale è bloccata.");
}
```

21.7. Buone Pratiche e Note Aggiuntive

21.7.1. Accedere ai Dettagli dell'Eccezione

Nel blocco catch, si può usare la variabile dell'eccezione per ottenere informazioni utili, come il messaggio di errore.

```
try {  
    throw new Exception("Operazione non valida");  
} catch (Exception e) {  
    String s = e.getMessage(); // Recupero il messaggio  
    System.err.println(s);     // Stampo il messaggio sullo standard error  
}
```

Integrazione: Oltre a `getMessage()`, un altro metodo molto utile è `printStackTrace()`, che stampa sull'errore standard l'intera traccia dello stack (la sequenza di chiamate di metodi che ha portato all'errore). È estremamente utile in fase di debug.

```
// Esempio  
// catch (Exception e) {  
//     e.printStackTrace();  
// }
```

21.7.2. Non Lasciare Blocchi catch Vuoti

Attenzione: Non lasciare mai vuoti i blocchi catch!

Un blocco catch vuoto "mangia" l'eccezione (EAT), nascondendo il problema. Questo rende il debugging estremamente difficile e può portare a comportamenti anomali e imprevedibili nel programma. Se si ritiene di non dover gestire un'eccezione, è buona pratica inserire almeno un commento che spieghi il perché.

21.7.3. Integrazione: Il Costrutto *try-with-resources*

Da Java 7 in poi, è stato introdotto un modo più semplice e sicuro per gestire le risorse (come file, stream, connessioni) che devono essere chiuse dopo l'uso. Si tratta del costrutto *try-with-resources*, che garantisce la chiusura automatica delle risorse, rendendo il blocco *finally* non più necessario per questo scopo.

L'esempio del `FileWriter` visto prima può essere riscritto in modo molto più conciso:

```
import java.io.*;

class MainModerno {
    public static void main(String[] args) {
        // La risorsa (Writer) viene dichiarata nel try
        try (Writer w = new FileWriter("out.txt")) {
            for (int i = 0; i < 10; i++) {
                w.write(String.valueOf(i)
                    + System.getProperty("line.separator"));
            }
        } catch (IOException ioe) {
            // w.close() viene chiamato automaticamente alla fine del try,
            // sia in caso di successo che di errore.
            System.out.println("Errore: " + ioe.getMessage());
        }
    }
}
```

Questo approccio è preferibile perché riduce la verbosità e previene bug legati a risorse non chiuse correttamente.

22. Asserzioni (Assertions)

22.1. Scopo e Utilizzo

Le asserzioni sono uno strumento utile durante la fase di debugging. Ci permettono di testare condizioni che noi sappiamo essere vere.

22.2. Sintassi

In Java, le asserzioni possono avere due forme:

1. Asserzione Semplice:

```
assert espressione_booleana;
```

2. Asserzione con Messaggio di Dettaglio:

```
assert espressione_booleana : altra_espressione;
```

L'espressione_booleana rappresenta la condizione che noi siamo certi debba essere vera. Ad esempio, se sappiamo che la variabile x deve essere uguale a y, possiamo scrivere:

```
assert x == y;
```

22.3. Comportamento

Se il risultato dell'espressione_booleana è **falso**, viene lanciato un `AssertionError`.

Se è presente `altra_espressione` (la seconda forma), il suo valore viene trasportato dall'`AssertionError` per fornire maggiori dettagli sull'errore.

```
assert x == y : "x vale " + x + " e y vale " + y;
```

22.4. Esempi d'Uso

Esempio 1: Verifica di una condizione post-operazione Possiamo usare un'asserzione per verificare che lo stato del sistema sia consistente dopo un'operazione.

```
double totale = getTotale();
cb1.trasferisci(cb2, 100.01);
double totaleDopo = getTotale();
assert totale == totaleDopo : totale + " " + totaleDopo;
```

Esempio 2: Ramo di codice non raggiungibile È una buona pratica inserire un'assert `false` in un ramo di codice che si ritiene non debba mai essere raggiunto.

```
// In un blocco switch, se tutti i casi possibili sono coperti
switch (valore) {
    case A:
        // ...
        break;
    case B:
        // ...
        break;
    default:
        assert false : "Questo caso non dovrebbe mai essere raggiunto!";
}
```

22.5. Abilitare le Asserzioni

Di default, la JVM esegue il codice con le **asserzioni disabilitate**. Per abilitarle, è necessario usare un'opzione da riga di comando all'avvio del programma:

- Forma breve: `java -ea NomeClasse`
- Forma estesa: `java -enableassertions NomeClasse`

22.6. Asserzioni vs. Eccezioni

Negli appunti viene fatta una distinzione tra Asserzioni ed Eccezioni, ma senza approfondire. Ecco una spiegazione integrativa:

Le asserzioni servono per individuare errori di logica interna during la fase di sviluppo e test. Verificano pre-condizioni, post-condizioni e invarianti, ovvero condizioni che devono **sempre** essere vere se il codice è corretto. Sono pensate per essere disattivate in produzione per non impattare sulle performance.

Le eccezioni (come `IllegalArgumentException`, `NullPointerException`, ecc.) gestiscono invece errori che possono verificarsi a runtime a causa di fattori esterni o input non validi (es. un file non trovato, un input errato dell'utente). Sono una parte fondamentale della gestione degli errori in un'applicazione robusta e sono sempre attive, sia in sviluppo che in produzione.

23. Programmazione concorrente

23.1. Thread

Questo argomento introduce la programmazione concorrente, quindi **quando farai l'indice questo argomento deve far parte del capitolo "Programmazione concorrente"**

23.1.1. Introduzione ai Thread

Un **thread** è un flusso di esecuzione indipendente. Possiamo avere più thread all'interno dello stesso processo.

Inserire qui il diagramma presente nella lezione 10. Edit: non è vitale e non ho voglia di inserirlo, se ti interessa vattelo a trovare nel pdf della lezione 10

Un processo può avere un singolo flusso di esecuzione (single-thread) o più flussi di esecuzione (multi-thread). Quando mandiamo in esecuzione un programma Java, la JVM crea un "main thread" che esegue il nostro codice (il metodo main). Questo thread principale può a sua volta generare altri thread.

Inserire qui i diagrammi sui thread. Edit: vedi sopra

Inserire qui la spiegazione sui diagrammi (rivedere videolezione 14/10/2025). Edit: non vedi che ho scritto sopra?

23.1.2. Creazione dei Thread

Tutti i thread sono rappresentati da istanze della classe `java.lang.Thread`. Esistono due modi principali per definire il comportamento di un thread.

23.1.2.1. Estendendo la Classe Thread

Il primo metodo consiste nel creare una sottoclasse di `Thread` che ridefinisce il metodo `run()`. Il codice all'interno di `run()` definisce il compito che il thread eseguirà.

Esempio di codice (UomoLupo.java) Questa classe definisce un thread che stampa una stringa 10.000 volte.

```
public class UomoLupo extends Thread {
    private String urlò;

    public UomoLupo(String s) {
        urlò = s;
    }

    public void run() {
        for (int i = 0; i < 10000; i++) {
            System.out.println(urlò);
        }
    }
}
```

```

    }
}

```

Esempio di codice (Main.java) Il metodo main crea due istanze di UomoLupo e le avvia.

```

class Main {
    public static void main(String[] args) {
        UomoLupo ul1 = new UomoLupo("Auuuuu!");
        UomoLupo ul2 = new UomoLupo("Ho fame...");

        ul1.start();
        ul2.start();
    }
}

```

(*1) All'interno del metodo run() definisco il comportamento dei Thread di tipo UomoLupo.

(*2) Per far partire un thread, dobbiamo chiamare il suo metodo start(). **NOTA:** non si deve chiamare direttamente il metodo run()! Chiamare run() eseguirebbe il codice nello stesso thread corrente, senza creare un nuovo flusso di esecuzione.

L'output del programma mostrerà un "intreccio" (interleaving) delle stampe dei due thread, poiché vengono eseguiti in modo concorrente. Il nostro processore **singole core, multi thread**, applica delle politiche di scheduling che eseguono il thread ul1 stampando "Auuuuu!" e ul2 stampando "Ho fame...", intrecciandoli di volta in volta

```

Auuuuu!
Auuuuu!
...
Auuuuu!
Ho fame...
Ho fame...
Ho fame...
...
Ho fame...
Auuuuu!
...
Auuuuu!
Ho fame...
Ho fame...
Auuuuu!
...

```

Nota bene: l'alternarsi dei thread non è perfetto, e il numero di stringhe consecutive stampate può variare in quanto i thread vengono eseguiti e alternati in intervalli infinitesimali di tempo. Qualora fosse

necessario, la stampa alternata (perfetta) delle stringhe, anche nel caso di single core multi thread, è comunque ottenibile attraverso alcuni vincoli di sincronizzazione tra i thread.

23.1.2.2 Implementando l'Interfaccia Runnable

Siccome in Java l'ereditarietà è singola, dover estendere la classe Thread può essere una forte limitazione. Per risolvere questo problema, è possibile seguire una strada alternativa che si basa sull'uso dell'interfaccia Runnable.

L'interfaccia Runnable è molto semplice:

```
public interface Runnable {  
    void run();  
}
```

La procedura è la seguente:

1. Si crea una classe che implementa Runnable e definisce il suo metodo run().
2. Si crea un'istanza di questa classe.
3. Si passa questo oggetto al costruttore della classe Thread.
4. Si fa partire il flusso di esecuzione invocando il metodo start() sull'oggetto Thread.

Esempio di codice (ComportamentoMostro.java)

```
public class ComportamentoMostro implements Runnable {
    private String nome;

    public ComportamentoMostro(String n) {
        nome = n;
    }

    public void run() {
        for (int i = 0; i < 10000; i++) {
            System.out.println(nome + ": Auagagaaaaaah");
            // Se metto yield(), il comportamento è "educato"
            // Thread.yield()
        }
    }
}
```

Esempio di codice (Main.java) Creiamo gli oggetti Thread passando loro le istanze di ComportamentoMostro.

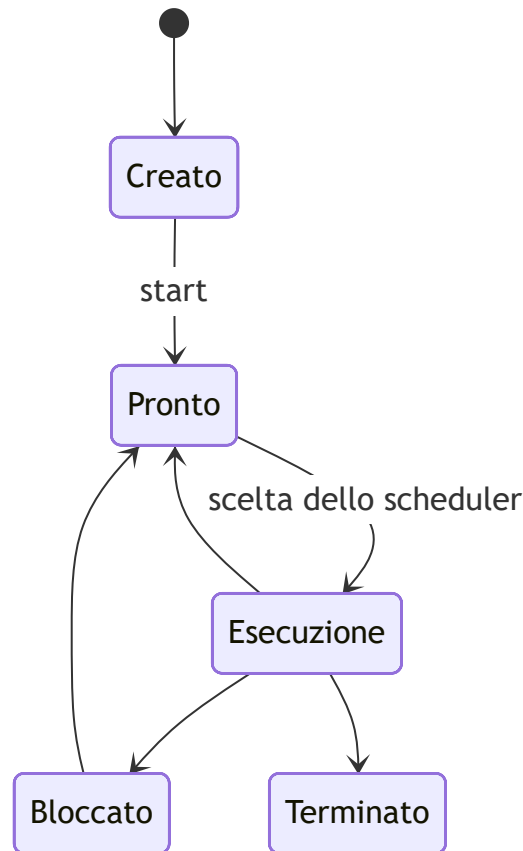
```
class Main {
    public static void main(String[] args) {
        ComportamentoMostro cz1 = new ComportamentoMostro("Uomo lupo");
        ComportamentoMostro cz2 = new ComportamentoMostro("Zombie");

        Thread t1 = new Thread(cz1);
        Thread t2 = new Thread(cz2);

        t1.start();
        t2.start();
    }
}
```

23.1.3. Ciclo di Vita di un Thread: Gli Stati

Un thread attraversa diversi stati durante la sua esistenza.



- **Creato (New):** L'oggetto Thread è stato istanziato, ma il metodo `start()` non è ancora stato invocato.
- **Pronto (Runnable):** Il thread è stato avviato (`start()` è stato chiamato) ed è in attesa che lo scheduler gli assegni il processore per l'esecuzione.
- **In Esecuzione (Running):** Lo scheduler ha assegnato il processore al thread, che sta eseguendo il suo codice. Un thread può lasciare questo stato per:
 - **Fine tempo:** è finito il tempo a disposizione del thread.
 - **Preemption (prelazione):** Lo scheduler decide di dare il processore a un altro thread (es. è pronto un thread con priorità maggiore).
 - **Cessione volontaria:** Il thread invoca `yield()`, cedendo volontariamente il processore.
- **Bloccato (Blocked/Waiting):** Il thread è temporaneamente inattivo e non può essere eseguito, perché in attesa di un evento. Cause comuni:
 - Invocazione di `sleep()` per attendere un certo tempo.
 - Attesa per acquisire un lock synchronized. Il thread rimane bloccato finché un'altro thread rilascia il lock.
 - Invocazione di `wait()`. Una volta che l'evento si verifica, il thread torna allo stato **Pronto**. Per sbloccarlo un altro thread deve fare una `notify()`.
 - Attesa per il completamento di un'operazione di I/O.
- **Terminato (Terminated):** Il thread ha concluso la sua esecuzione perché il metodo `run()` è terminato. Un thread terminato non può essere riavviato.

23.1.4. Thread e Garbage Collection

Gli oggetti thread non sono soggetti a garbage collection (fino a che sono in vita) anche se non conserviamo il riferimento ad essi.

Il processo di garbage collection considera raggiungibili tutti gli oggetti nello heap usando come punti di partenza gli stack di tutti i thread in vita. Ogni thread ha il suo stack.

23.2. Altro sui Thread

La JVM mantiene il processo in vita finché almeno un thread è in esecuzione. Quando tutti i thread hanno terminato la loro esecuzione (cioè sono arrivati in fondo al loro metodo `run()`), la JVM termina. Tuttavia, è possibile forzare la terminazione immediata della JVM e di tutti i suoi thread chiamando il metodo `System.exit()`.

23.2.1. Ciclo di Vita di un Thread

Nota Importante: Se un thread termina la propria esecuzione, non può essere fatto ripartire chiamando il metodo `start()` una seconda volta. È necessario creare (con `new`) un altro oggetto thread.

23.2.2. Creazione ed Esecuzione di un Thread

Per creare un thread, è possibile estendere la classe `Thread` e fare l'override del suo metodo `run()`, che conterrà il codice che il thread dovrà eseguire.

Esempio:

```
public class MioThread extends Thread {  
    public void run() {  
        for(int i = 0; i < 100; i++) {  
            System.out.println("Ciao");  
        }  
    }  
}
```

Per eseguire il thread, è necessario creare un'istanza della classe e invocare il metodo `start()`.

```
public class Prova {  
    public static void main(String[] args) {  
        MioThread t1 = new MioThread();  
        MioThread t2 = new MioThread();  
  
        t1.start();  
        t2.start();  
    }  
}
```

```

    }
}

```

Nota bene: l'esecuzione di `t1.start()` e `t2.start()` non causa una terminazione della JVM. La JVM termina la propria esecuzione quando entrambi i thread arrivano in fondo al loro metodo `run()`.

23.3. Proprietà e Gestione dei Thread

23.3.1. Naming dei Thread

I thread hanno un **nome** che può essere utile solo per scopi di debugging.

- **Impostazione del nome:** Tramite il costruttore `Thread(String name)`.
- **Recupero del nome:** Tramite il metodo `String getName()`.

Se non viene specificato un nome, la JVM ne assegna uno predefinito (es. "Thread-0", "Thread-1", ecc.).

23.1.2. Priorità e Scheduling

I thread hanno un **livello di priorità**, un valore intero che influenza le decisioni dello scheduler.

- `Thread.MIN_PRIORITY`: Valore 1.
- `Thread.MAX_PRIORITY`: Valore 10.
- `Thread.NORM_PRIORITY`: Valore 5, che è anche il livello predefinito del main thread.

Quando viene creato un nuovo thread, il suo livello di priorità viene "ereditato" dal thread padre. È possibile modificare la priorità con il metodo `setPriority()`.

La politica di scheduling è **fixed-priority**: lo scheduler sceglie, tra i thread pronti, quello con la priorità più alta. Questo sistema implementa la **preemption**: se un thread con priorità più alta diventa pronto, il processore viene revocato al thread attualmente in esecuzione e assegnato a quello con priorità maggiore.

Se sono pronti più thread con lo stesso livello ne viene scelto uno a caso.

23.1.3. Flussi di esecuzione e oggetti

Qui sotto ci sono due esempi: il primo mostra un thread che lavora su un oggetto privato e non condivide stato col thread principale; il secondo introduce un campo condiviso tra thread e main per far emergere i problemi di accesso concorrente.

Esempio Semplice: Thread con Oggetto Privato

```

class A {

    void m(){
        for(int i = 0; i < 100; i++) {
            System.out.println("ABC");
        }
    }
}

```

```

    }
}

class ProvaThread extends Thread {

    public void run(){
        A a1 = new A();
        a1.m();
    }
}

ProvaThread p1 = new ProvaThread();
p1.start();

```

Esempio con Stato Condiviso tra Thread e Main

```

class AltroThread extends Thread {
    int x;

    void incrementa() { // viene eseguito dal main thread
        x++;
    }

    public void run(){
        for(int i = 0; i < 100; i++) {
            System.out.println("XYZ");
        }
    }
}

// nel main:

AltroThread t1 = new AltroThread();
t1.start();
t1.incrementa();

```

Il primo esempio illustra un thread che crea e usa un oggetto locale a sé stesso, quindi non condivide stato con il thread principale ed è utile per capire il comportamento di un thread isolato.

Il secondo esempio, invece, mostra un campo (x) nell'oggetto thread che può essere accessibile sia dal thread creato (quando esegue `run()`) sia dal main (quando chiama `incrementa()`): questo introduce la possibilità di accesso concorrente allo stesso dato.

La differenza tra i due casi evidenzia perché la condivisione di stato può portare a inconsistenze (race conditions) se non si adottano meccanismi di protezione.

23.4. Metodi Utili della Classe Thread

23.4.1. Il Metodo sleep()

Il metodo static void sleep(long millis) sospende l'esecuzione del thread corrente per il numero specificato di millisecondi. Si può presentare in due forme:

- static void sleep(long millis) throws InterruptedException: sospende il thread per il numero specificato di millisecondi.
- static void sleep(long millis, int nanos) throws InterruptedException: sospende il thread per il numero specificato di millisecondi e nanosecondi.

Questo metodo è utile per simulare ritardi o per cedere volontariamente il controllo della CPU, permettendo ad altri thread di essere eseguiti. Poiché può lanciare una InterruptedException (un'eccezione di tipo "controllato"), la sua chiamata deve essere obbligatoriamente racchiusa in un blocco try-catch.

Esempio con sleep():

```
public class ThreadConSleep extends Thread {
    private String nome;
    private long quanto;

    public ThreadConSleep(long ms) {
        quanto = ms;
        nome = getName(); // Ereditato da Thread
    }

    public void run() {
        for (int i = 0; i < 100; i++) {
            try {
                System.out.println(nome + ": " + i);
                sleep(quanto); // Ereditato da Thread
            } catch (InterruptedException ie) {
                System.out.println("Sono stato interrotto...");
                return;
            }
        }
    }
}

class Main {
    public static void main(String[] args) {
        ThreadConSleep t1 = new ThreadConSleep(300);
        ThreadConSleep t2 = new ThreadConSleep(500);
        t1.start();
        t2.start();
    }
}
```

```

    }
}

```

Il blocco try-catch è necessario perché `sleep()` può essere interrotto da un altro thread che chiama il metodo `interrupt()` sul thread in questione. In tal caso, viene lanciata un'`InterruptedException`, che deve essere gestita in quanto è un'eccezione controllata.

23.4.2. Il Metodo `currentThread()`

Se si vuole invocare `sleep()` da una classe che non estende `Thread`, è necessario ottenere un riferimento al thread corrente. Questo è possibile tramite il metodo statico `Thread.currentThread()`, che appunto restituisce il riferimento al thread che sta eseguendo il metodo.

Esempio:

```

class Main {
    public static void main(String[] args) {
        Contatore c1 = new Contatore(300);
        ThreadCheConta t1 = new ThreadCheConta(c1);
        Contatore c2 = new Contatore(500);
        ThreadCheConta t2 = new ThreadCheConta(c2);
        t1.start();
        t2.start();
    }
}

public class Contatore {
    private long quanto;

    public Contatore(long q) {
        this.quanto = q;
    }

    public void stampa() {
        for (int i = 0; i < 100; i++) {
            try {
                // Ottiene il riferimento al thread che esegue questa istruzione
                Thread t = Thread.currentThread();
                String n = t.getName(); // Ottiene il nome del thread
                System.out.println(n + ": " + i);

                // Invoca sleep sulla classe Thread
                Thread.sleep(quanto);
            } catch (InterruptedException ie) {
                System.out.println("Sono stato interrotto...");
            }
        }
    }
}

```

```
        return;  
    }  
}  
}
```

Nota bene: `Thread.sleep()` sospende il thread corrente, non il thread su cui viene chiamato il metodo. Per questo motivo, è un metodo statico. Non mi trovo in una sottoclasse di `Thread`, quindi non posso chiamare `sleep()` direttamente; devo usare `Thread.sleep()`.

```
public class ThreadCheConta extends Thread {  
    private Contatore c;  
  
    public ThreadCheConta(Contatore c) {  
        this.c = c;  
    }  
  
    public void run() {  
        c.stampa();  
    }  
}
```

23.5. Programmazione Concorrente e Corse Critiche

Quando più thread accedono e modificano dati condivisi, possono sorgere problemi di consistenza. Questa situazione è nota come "corsa critica" (race condition).

23.5.1. Il Problema dell'Inconsistenza dei Dati

Consideriamo un oggetto ContoBancario condiviso tra più thread Depositatore. Ogni thread esegue un ciclo infinito di depositi.

```
public class ContoBancario {
    int bilancio;
    int numOperazioni;

    void deposita() {
        bilancio += 10;
        numOperazioni++;
        assert numOperazioni * 10 == bilancio : "numOperazioni="
            + numOperazioni + " bilancio=" + bilancio;
    }
}

public class Depositatore extends Thread {
    ContoBancario c;

    Depositatore(ContoBancario c) {
        this.c = c;
    }

    public void run() {
        while (true) {
            c.deposita();
        }
    }
}

public class Main {
    public static void main(String[] args) {
        ContoBancario conto = new ContoBancario();
        Depositatore d1 = new Depositatore(conto);
        Depositatore d2 = new Depositatore(conto);
        Depositatore d3 = new Depositatore(conto);
        d1.start();
        d2.start();
        d3.start();
    }
}
```

```

    }
}

```

L'esecuzione concorrente di `deposita()` può portare l'oggetto `ContoBancario` in uno stato inconsistente, violando l'invariante `bilancio == numOperazioni * 10` e causando un `AssertionError`.

Supponiamo di avere due thread `t1` e `t2` che `bilancio = 1000` e `numOperazioni = 100`.

Inserire schema lezione 11. Edit: vattelo a vedere non è vitale

Il risultato finale dovrebbe essere `bilancio = 1020` e `numOperazioni = 102`, ma a causa della concorrenza, potrebbe risultare in `bilancio = 1010` e `numOperazioni = 102`, violando l'invariante.

23.5.2. Operazioni non Atomiche

I problemi visti prima derivano dal fatto che l'operazione `deposita()` **non è atomica**. *Un'operazione non atomica è composta da più passaggi distinti (es. leggi valore, modifica valore, scrivi valore).*

Un thread può essere descheduled (perdere la CPU) a metà dell'operazione, lasciando l'oggetto in uno stato inconsistente. Se un altro thread esegue la stessa operazione nel frattempo, può leggere i dati "vecchi", portando a un risultato finale errato, come una "lost update".

Ci sono anche problemi che non siamo in grado di rilevare, per esempio:

Inserire lo schema. Edit: si dovrebbe essere capito adesso no?

```

bilancio = 1010
numOperazioni = 101

```

Ci sono state due operazioni di deposito. Il valore di `bilancio` doveva essere 1020 e quello di `numOperazioni` doveva essere 102, ma a causa della concorrenza, il valore di `numOperazioni` è rimasto 101.

L'assert che abbiamo inserito esprime un vincolo relativo al singolo oggetto `ContoBancario`. Non vengono espressi altri vincoli relativi al numero di operazioni eseguite e a quante volte `deposita()` è stato chiamato. Quindi, anche se l'assert non viene violato, l'oggetto può comunque trovarsi in uno stato inconsistente.

23.6. Sincronizzazione e Mutua Esclusione

In Java, il problema delle corse critiche viene risolto con il meccanismo dei **monitor**.

23.6.1. Il Meccanismo dei Monitor in Java

■ Ogni oggetto in Java è associato a un **lock** (o "lucchetto"). Un lock può essere "aperto" o "chiuso".

Un thread, per poter eseguire un metodo `synchronized` su un oggetto, deve prima acquisire il lock di quell'oggetto. Se il lock è già posseduto da un altro thread, il primo thread deve attendere che il secondo lo rilasci.

23.6.2. La keyword `synchronized`

Quando un thread esegue un metodo dichiarato `synchronized` su un oggetto `x`:

1. **Acquisisce** il lock di `x`.
2. **Esegue** il corpo del metodo.
3. **Rilascia** il lock di `x`.

Questo garantisce che il metodo venga eseguito in **mutua esclusione**: un solo thread alla volta può eseguire un metodo `synchronized` su una data istanza di un oggetto.

23.6.3. Esempio Corretto con `synchronized`

Per risolvere il problema del ContoBancario, è sufficiente dichiarare il metodo `deposita()` come `synchronized`.

```
public class ContoBancario {  
    int bilancio;  
    int numOperazioni;  
  
    // Ora il metodo è atomico e thread-safe  
    synchronized void deposita() {  
        bilancio += 10;  
        numOperazioni++;  
        assert numOperazioni * 10 == bilancio : "numOperazioni=" +  
            numOperazioni + " bilancio=" + bilancio;  
    }  
}
```

Quando un Depositatore chiama `deposita()`, deve per prima cosa ottenere il lock sull'oggetto `conto` e solo dopo passa a modificare l'oggetto. Al termine del metodo, il lock viene rilasciato in modo che possa essere acquisito da altri Depositatore.

In pratica abbiamo un'esecuzione in **mutua esclusione** del metodo `deposita()`.

Se la classe fosse stata un pò più complessa, per esempio:

```
public class ContoBancario2 {  
    int numOp;  
    int blancio;  
    String intestatario;  
  
    public ContoBancario2(String s) {  
        intestatario = s;  
    }  
  
    public synchronized void deposita() {    // synchronized  
        bilancio += 10;  
        numOp++;  
    }  
  
    public synchronized void preleva() {    // synchronized  
        bilancio -= 10;  
        numOp++;  
    }  
  
    public String getIntestatario() {    // non synchronized  
        return intestatario;  
    }  
}
```

Otteniamo così **mutua esclusione** deposita-preleva, deposita-deposita, preleva-preleva. Invece non abbiamo la mutua esclusione tra getIntestatario() e preleva o deposita.

23.7. Il Problema dell'Interferenza

Quando più thread accedono e modificano dati condivisi, può verificarsi un'interferenza che porta a uno stato inconsistente del programma. La **mutua esclusione** è un meccanismo che garantisce che solo un thread alla volta possa eseguire una sezione di codice critica, prevenendo così le race condition.

23.7.1. Metodi synchronized

In Java, il modo più semplice per ottenere la mutua esclusione è dichiarare un metodo come `synchronized`.

Definizione: Un metodo `synchronized` acquisisce il **lock** intrinseco dell'oggetto su cui viene invocato prima di eseguire il suo corpo. Il lock viene rilasciato quando il metodo termina.

Finché un thread possiede il lock di un oggetto, nessun altro thread può eseguire un metodo `synchronized` sullo **stesso oggetto**.

- **Lock diversi:** Thread diversi possono eseguire contemporaneamente metodi `synchronized` su **oggetti diversi**, perché ogni oggetto ha il suo lock.

```
// Supponiamo di avere due oggetti ContoBancario
ContoBancario cb1 = new ContoBancario(); // Oggetto 1 con Lock 1
ContoBancario cb2 = new ContoBancario(); // Oggetto 2 con Lock 2

// t1 acquisisce il lock di cb1
// t1:
cb1.preleva();

// t2 può eseguire preleva() contemporaneamente
// perché cb2 è un oggetto diverso e ha un lock diverso.
// t2:
cb2.preleva();
```

23.7.2. Lock Rientranti

I lock di Java sono **rientranti**.

Definizione: Se un thread possiede già il lock di un oggetto, può richiederlo nuovamente senza bloccarsi.

Questo è fondamentale per permettere a un metodo `synchronized` di invocarne un altro sulla stessa istanza senza causare un deadlock.

```
public class A {  
    // m1 acquisisce il lock su 'this'  
    public synchronized void m1(){  
        // ...  
        m2();  
        // ...  
        // il lock viene rilasciato al termine di m1()  
    }  
  
    private synchronized void m2(){  
        // L'acquisizione del lock avviene senza problemi.  
    }  
}
```

23.7.3. Rilascio del Lock

Nel caso di un metodo `synchronized`, il lock viene rilasciato automaticamente in ogni circostanza in cui il thread lascia il metodo:

- **Esecuzione normale:** quando termina l'esecuzione del corpo del metodo.
- **Eventi anomali:** quando il metodo termina a causa di un'eccezione non gestita.

23.8. Blocchi synchronized

A volte, sincronizzare un intero metodo è troppo restrittivo. I blocchi `synchronized` offrono un controllo più granulare sulla mutua esclusione.

23.8.1 Sintassi e Funzionamento

La sintassi permette di specificare esplicitamente su quale oggetto si desidera acquisire il lock.

```
synchronized (oggetto) { // Acquisisce il lock di 'oggetto'
    //
    // Blocco di istruzioni protetto dal lock in questione
    //
}
```

23.8.2. Equivalenza con i Metodi `synchronized`

Un metodo `synchronized` è sintatticamente equivalente a un metodo il cui intero corpo è racchiuso in un blocco `synchronized(this)`. La forma con il metodo `synchronized` è da preferire quando applicabile, perché più **sintetica** e **leggibile**.

```
// QUESTA FORMA (leggibile e sintetica, quindi preferibile)
class B {
    synchronized void m1(){
        // ...
    }
    synchronized void m2(){
        // ...
    }
}

// È EQUIVALENTE A QUESTA
class B {
    void m1() {
        synchronized (this) {
            // ...
        }
    }
    void m2() {
        synchronized (this) {
            // ...
        }
    }
}
```

23.8.3. Quando Usare i Blocchi `synchronized`

Ci sono situazioni in cui i blocchi `synchronized` sono necessari:

1. **Mutua esclusione parziale:** non tutto il codice di un metodo deve essere eseguito in mutua esclusione.

```
public void m() {  
    // Codice eseguito in parallelo al altri thread  
    // ...  
    synchronized (this) {  
        // Solo questo blocco è in mutua esclusione  
    }  
    // Altro codice eseguito in parallelo  
}
```

2. **Sincronizzazione su un oggetto diverso:** è necessario sincronizzare su un oggetto diverso da quello implicito.

```
public void m(Parametro p) {  
    // ...  
    synchronized (p) {  
        // Si prende il lock di p  
        // Codice che modifica p  
    }  
    // ...  
}
```

3. **Sincronizzazione su un array:** per proteggere sezioni critiche che leggono/modificano l'array. Attenzione: sincronizzare sull'array non sincronizza automaticamente l'accesso ad ogni singolo elemento se si usano riferimenti diversi all'array o si sostituisce l'array.

```
// esempio: sincronizzare sull'array stesso  
final String[] arr = new String[100];  
synchronized (arr) {  
    arr[0] = "x";  
}
```

23.8.4. Sincronizzazione su Lock Multipli per Maggiore Parallelismo

L'uso di oggetti di lock separati per sezioni di dati indipendenti può **aumentare il livello di parallelismo** del codice. In questo modo, i thread che operano su dati diversi non si bloccano a vicenda.

Il seguente esempio riporta più lock, diversi dall'oggetto implicito.

```
class Separati {
    private int v1;
    private int v2;
    // Oggetti usati solo come lock
    protected final Object lock1 = new Object();
    protected final Object lock2 = new Object();

    // Questo metodo usa solo lock1
    public int getV1() {
        synchronized (lock1) {
            return v1;
        }
    }

    // Anche questo metodo usa solo lock1
    public void setV1(int v) {
        synchronized (lock1) {
            v1 = v;
        }
    }

    // Questo metodo usa solo lock2
    public int getV2(){
        synchronized (lock2){
            return v2;
        }
    }

    // Questo metodo usa solo lock2
    public void setV2(int v) {
        synchronized (lock2) {
            v2 = v;
        }
    }

    // Questo metodo ha bisogno di entrambi i lock
    public void makeEqual(){
        synchronized (lock1) {
            synchronized (lock2) {
```

```
        v1 = v2;  
    }  
}  
}
```

Con questa implementazione, ottengo che una chiamata a:

- `getV1()` può essere eseguita "in contemporanea" con `getV2()` o `setV2()` (e viceversa), perché operano su lock distinti
- `setV1()` può essere eseguita "in contemporanea" con `getV2()` o `setV2()`
- si può fare la stessa cosa a parti invertite
- c'è invece mutua esclusione `getV1()` e `setV1()` e `makeEqual()`
- c'è mutua esclusione `getV2()` e `setV2()` e `makeEqual()`

Tutto questo aumenta il livello di **parallelismo** nel codice.

23.9. Metodi Statici Sincronizzati

23.9.1. Il Lock sulla Classe

Definizione: Quando un metodo static è dichiarato `synchronized`, il lock che viene acquisito è quello dell'oggetto `Class` associato alla classe, e non quello di una specifica istanza.

Questo significa che **due thread non possono eseguire contemporaneamente metodi statici sincronizzati sulla stessa classe.**

23.9.2. Esempio Pratico: Contatore Statico

Consideriamo una classe `ContoBancario` che deve assegnare un numero di conto univoco e progressivo a ogni nuova istanza.

```
// VERSIONE NON SICURA (rischio di race condition)
public class ContoBancario {
    private int numero;
    private static int prossimoNumero;

    public ContoBancario() {
        numero = prossimoNumero++;
    }
}
```

Se più thread eseguono questo costruttore contemporaneamente, potrebbero leggere lo stesso valore di `prossimoNumero` e assegnare numeri di conto duplicati, portando a inconsistenze.

La soluzione è accedere al campo statico `prossimoNumero` in modo sincronizzato, usando un metodo statico.

```
// VERSIONE SICURA
public class ContoBancario {
    private int numero;
    private static int prossimoNumero;

    public ContoBancario() {
        // L'assegnazione ora è atomica
        numero = getProssimoNumero();
    }

    // Acquisisce il lock della classe ContoBancario.class
    private static synchronized int getProssimoNumero(){
        return prossimoNumero++;
    }
}
```

```
    }
}
```

Prendendo ad esempio:

```
// t1:
ContoBancario c1 = new ContoBancario();
// t2:
ContoBancario c2 = new ContoBancario();
```

Si può notare che ci sono tre lock.

23.10. Deadlock

23.10.1. Definizione ed Esempio Teorico

Definizione: Un **deadlock** è una situazione di blocco da cui due o più thread non riescono più a uscire.

Si verifica tipicamente quando due thread acquisiscono lock su risorse diverse in ordine inverso, bloccandosi a vicenda.

Scenario Classico:

1. **Thread t1:** acquisisce lock1 e subito dopo tenta di acquisire lock2.
2. **Thread t2:** acquisisce lock2 e subito dopo tenta di acquisire lock1.

t1 rimane bloccato in attesa di lock2 (posseduto da t2), e t2 rimane bloccato in attesa di lock1 (posseduto da t1). Nessuno dei due può proseguire.

23.10.2. Esempio Pratico di Deadlock in Java

Il seguente scenario con due thread, Mario e Luigi, che interagiscono con due oggetti condivisi, StatoGioco e ViteRimaste, può portare a un deadlock.

```
// CLASSE Lancia (Lancia.java)

public class Lancia {
    public static void main(String[] args) {
        StatoGioco s = new StatoGioco();
        ViteRimaste v = new ViteRimaste();
        Mario m = new Mario(s,v);
        Luigi l = new Luigi(s, v);
        m.start();
        l.start();
    }
}
```

```
// THREAD Luigi (Luigi.java)
```

```
public class Luigi extends Thread {  
  
    private StatoGioco sg;  
    private ViteRimaste vr;  
  
    public Luigi(StatoGioco sg, ViteRimaste vr) {  
        this.sg = sg;  
        this.vr = vr;  
    }  
    public void run(){  
        while(true) {  
            // altro codice  
            vr.controllaQuante(sg);  
            // Lock ViteRimaste --> Lock StatoGioco  
        }  
    }  
}
```

```
// THREAD Mario (Mario.java)
```

```
public class Mario extends Thread {  
  
    private StatoGioco sg;  
    private ViteRimaste vr;  
  
    public Mario(StatoGioco sg, ViteRimaste vr) {  
        this.sg = sg;  
        this.vr = vr;  
    }  
    public void run(){  
        while(true) {  
            // altro codice  
            sg.controllaPunteggio(vr);  
            // Lock StatoGioco --> Lock ViteRimaste  
        }  
    }  
}
```

```
// CLASSE StatoGioco (StatoGioco.java)
```

```
public class StatoGioco {  
  
    private boolean fine;  
  
    public synchronized void controllaPunteggio(ViteRimaste vr) {  
        System.out.println(Thread.currentThread().getName()  
            + " chiamato getPunteggio()");  
    }  
}
```

```

        // supponiamo che questo metodo venga chiamato sempre
        // quando il punteggio supera una certa soglia

        // Quindi, ora tenta di acquisire il lock di ViteRimaste
        vr.incrementa();
    }

    public synchronized void termina(){
        fine = true;
        System.out.println(Thread.currentThread().getName()
            + " chiamato termina()");
    }
}

// CLASSE ViteRimaste (ViteRimaste.java)
public class ViteRimaste {

    private int q;

    public synchronized void incrementa(){
        // supponiamo anche qui del codice opportuno
        System.out.println(Thread.currentThread().getName()
            + " chiamato incrementa()");
    }

    public synchronized void controllaQuante(StatoGioco sg) {
        System.out.println(Thread.currentThread().getName()
            + " chiamato quante()");
        // termina() potrebbe essere chiamato quando Le vite finiscono

        // Ora tenta di acquisire il lock di StatoGioco
        sg.termina();
    }
}

```

Il caso trattato, in senso generico, è quello di voler eseguire l'operazione B in t2 solo dopo che t1 ha eseguito A.

Se Mario esegue controllaPunteggio e acquisisce il lock di sg, e contemporaneamente Luigi esegue controllaQuante e acquisisce il lock di vr, si crea un deadlock quando entrambi tentano di acquisire il lock dell'altro oggetto.

23.11. Comunicazione tra Thread: wait(), notify() e notifyAll()

23.11.1. Oltre la Mutua Esclusione

Mentre `synchronized` risolve i problemi di interferenza, mediante esecuzione in mutua esclusione, i problemi di **comunicazione** (o meglio, sincronizzazione condizionale, nel mondo della programmazione concorrente) vengono risolti con i metodi `wait()`, `notify()` e `notifyAll()`. Questi metodi, ereditati dalla classe `Object`, sono disponibili per tutti gli oggetti Java.

Permettono a un thread di sospendere la propria esecuzione in attesa che una certa condizione si verifichi.

23.11.2. I Metodi Fondamentali e il "Wait-Set"

A ogni oggetto è associato un **"wait-set"**, ovvero un insieme di thread che sono in attesa su quell'oggetto.

- `public final void wait() throws InterruptedException`
 - Il thread che invoca `wait()` **sospende la propria esecuzione**.
 - **Rilascia il lock** sull'oggetto, permettendo ad altri thread di acquisirlo.
 - Viene inserito nel wait-set dell'oggetto.
 - Quando esce dall'attesa, **riacquisisce automaticamente il lock** prima di continuare.
- `public final void notify()`
 - Risveglia **un** thread a caso tra quelli bloccati nel wait-set dell'oggetto.
- `public final void notifyAll()`
 - Risveglia **tutti** i thread bloccati nel wait-set dell'oggetto.

Nota: Esistono anche versioni di `wait()` con timeout, come `wait(long timeout)` o `wait(long timeout, int nano)`, che sbloccano il thread dopo il tempo specificato (millisecondi) se non viene notificato prima. `wait()` è equivalente a `wait(0)`, che indica un timeout infinito. Il thread si blocca fino a che:

- `notify()` invocata su questo oggetto e il thread è selezionato (a caso)
- `notifyAll()` invocata su questo oggetto
- scatta il timeout (millisecondi)
- invocato il metodo `interrupt()` sul thread

23.11.3. Regole di Utilizzo e `IllegalMonitorStateException`

Regola Fondamentale: Per poter chiamare i metodi `wait()`, `notify()` o `notifyAll()` su un oggetto, il thread corrente **deve possedere il lock di tale oggetto**.

Se questa regola non viene rispettata, viene lanciata un'eccezione di tipo `IllegalMonitorStateException`.

Questo significa che questi metodi devono quasi sempre essere chiamati dall'interno di un blocco o di un metodo `synchronized` sull'oggetto in questione.

23.11.4. Schema di Utilizzo Tipico

Lo schema corretto per attendere una condizione prevede un ciclo `while`.

```
public class X {  
  
    private boolean condizione = false;  
  
    synchronized void faiQuandoCondizione() throws InterruptedException {  
        while (!condizione) {  
            wait();  
            // Quando il thread si risveglia E la condizione è vera,  
            // esegue il resto del codice.  
        }  
  
        synchronized void cambiaCondizione() {  
            // ... codice che rende 'condizione' vera ...  
            // Notifica ai thread in attesa che la condizione potrebbe  
            // essere cambiata e che possono ricontrollare.  
            notify();    // o notifyAll()  
        }  
    }  
}
```

Nota: l'uso di un ciclo `while` invece di un `if` è cruciale. Protegge da "risvegli spuri" (un thread può svegliarsi da `wait()` senza essere stato notificato) e garantisce che la condizione sia effettivamente vera prima di procedere, specialmente quando più thread sono in attesa.

23.12. Caso di Studio: Il Problema dei Produttori e Consumatori

Questo è un classico problema di sincronizzazione che modella un flusso di lavoro comune.

23.12.1. Descrizione del Problema

- **Produttori:** creano dati e li inseriscono in un'area di memoria condivisa (**buffer**).
- **Consumatori:** prelevano i dati dal buffer e li elaborano.
- **Buffer:** ha una capacità limitata ed è condiviso tra produttori e consumatori.

Vincoli di Sincronizzazione:

1. Un produttore deve attendere se il buffer è pieno.
2. Un consumatore deve attendere se il buffer è vuoto.

23.12.2. Implementazione della Classe Buffer

Il buffer gestisce la logica di inserimento ed estrazione in modo sicuro, utilizzando `wait()` e `notifyAll()` per coordinare i thread.

```
public class Buffer {
    private static final int DEFAULT_SIZE = 4;
    private String[] vett;
    private int testa, coda, quanti;

    public Buffer() {
        this(DEFAULT_SIZE);
    }

    public Buffer(int n) {
        vett = new String[n];
        testa = coda = quanti = 0;
    }

    public synchronized void inserisci(String s) throws InterruptedException {
        // Attende finché il buffer è pieno
        while (pieno()) {
            System.out.println(Thread.currentThread().getName()
                + ": mi blocco, buffer pieno");
            wait(); // Rilascia il lock e attende
        }

        vett[coda] = s;
        coda = (coda + 1) % vett.length;
        quanti++;

        // Notifica a tutti i thread (in particolare ai consumatori)
    }
}
```

```
// che un nuovo elemento è disponibile.
notifyAll();
}

public synchronized String estrai() throws InterruptedException {
    // Attende finché il buffer è vuoto
    while (vuoto()) {
        System.out.println(Thread.currentThread().getName()
            + ": mi blocco, buffer vuoto");
        wait(); // Rilascia il lock e attende
    }

    String t = vett[testa];
    testa = (testa + 1) % vett.length;
    quanti--;

    // Notifica a tutti i thread (in particolare ai produttori)
    // che uno spazio si è liberato.
    notifyAll();
    return t;
}

public synchronized boolean pieno() {
    return quanti == vett.length;
}

public synchronized boolean vuoto() {
    return quanti == 0;
}
}
```

23.12.3. Implementazione delle Classi Produttore e Consumatore

Queste classi estendono Thread e interagiscono con l'istanza condivisa di Buffer.

```
// CLASSE Produttore
public class Produttore extends Thread {

    private Buffer b;

    public Produttore(String n, Buffer b) {
        super(n);    // chiama il costruttore di Thread
        this.b = b;
    }

    public void run(){
        int c = 0;
        try {
            while(true) {
                sleep((long)(Math.random() * 1000));
                // Math.random() genera un double tra 0.0 e 1.0
                // Simula la produzione dormendo per t < 1 s
                b.inserisci(getName() + ": " + c++);
                System.out.println(getName() + ": ho inserito");
            }
        } catch (InterruptedException ie) {
            System.out.println("Sono stato interrotto");
            return;
        }
    }
}

// CLASSE Consumatore
public class Consumatore extends Thread {

    private Buffer b;

    public Consumatore(String n, Buffer b) {
        super(n);
        this.b = b;
    }

    public void run(){
        try {
            while(true) {
                sleep((long)(Math.random() * 1000));
```

```
        // Simula il consumo
        String s = b.estrai();
        System.out.println(getName() + ": ho estratto \"" + s + "\"");
    }
} catch (InterruptedException ie) {
    System.out.println("Sono stato interrotto");
    return;
}
}
}
```

23.12.4. Classe Principale di Avvio

Il main si occupa di creare il buffer, i produttori e i consumatori, e di avviarli.

```
import java.util.Scanner;

public class Prova {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Quanti produttori? ");
        int p = sc.nextInt();
        System.out.println("Quanti consumatori? ");
        int c = sc.nextInt();
        System.out.println("Dimensione del buffer? ");
        int b = sc.nextInt();

        Buffer buf = new Buffer(b);

        for (int i = 0; i < p; i++) {
            new Produttore("Produttore " + i, buf).start();
        }

        for (int i = 0; i < c; i++) {
            new Consumatore("Consumatore " + i, buf).start();
        }
    }
}
```

23.13. Gestione dell'Attesa: `notify()` vs `notifyAll()`

Più thread possono essere bloccati in `wait()` eseguito sullo stesso oggetto.

23.13.1. Il Pericolo di `notify()`

Se i thread sono in attesa di condizioni diverse, è necessario usare `notifyAll()`.

Se in tale situazione si usasse una semplice `notify()`, potrebbe succedere che:

1. Venga svegliato un solo thread.
2. Questo thread, però, non può andare avanti con la propria esecuzione (perché attendeva una condizione diversa da quella che si è appena verificata).
3. Il thread appena svegliato controlla la condizione e si riblocca.
4. Il thread che potrebbe andare avanti non è stato risvegliato.

Risultato finale: applicazione bloccata (potenziale deadlock).

23.13.2. L'Ottimizzazione `notify()`

Usare `notify()` al posto di `notifyAll()` è un'ottimizzazione possibile soltanto quando entrambe le seguenti condizioni sono vere:

- Tutti i thread bloccati attendono la stessa identica condizione.
- Solo uno dei thread può trarre beneficio dalla nuova situazione (es. un solo elemento è stato inserito, solo un thread può prenderlo).

23.13.3. Esempio: Deadlock nel Produttore-Consumatore

Cosa succederebbe se sostituissimo `notifyAll()` con `notify()` nel problema Produttore-Consumatore?

Scenario: 2 Produttori (P0, P1), 1 Consumatore (C0), Buffer con dimensione 1.

Possibile sequenza:

1. P0 inserisce un valore. (Buffer Pieno)
2. P1 prova a inserire -> si blocca (attende buffer non pieno).
3. P0 prova a inserire -> si blocca (attende buffer non pieno).
4. C0 estrae il valore (Buffer Vuoto) e risveglia (a caso) P0 (usando `notify()`).
5. C0 prova a estrarre -> si blocca (attende buffer non vuoto).
6. P0 (svegliato) inserisce un valore (Buffer Pieno) e risveglia (a caso) P1 (usando `notify()`).
7. P1 (svegliato) prova a inserire -> si blocca (Buffer Pieno).
8. P0 prova a inserire -> si blocca (Buffer Pieno).
9. DEADLOCK!

Spiegazione: si è arrivati a un punto in cui P0 e P1 sono bloccati in attesa che il buffer non sia pieno, e C0 è bloccato in attesa che il buffer non sia vuoto. L'errore è avvenuto al punto 6: P0 (Produttore) ha svegliato P1 (un altro Produttore), invece di svegliare C0 (il Consumatore). `notifyAll()` avrebbe svegliato tutti, incluso C0, che avrebbe trovato la sua condizione (buffer non vuoto) vera.

23.14. Esempio Pratico: Gestore di risorse equivalenti

23.14.1. Logica di Funzionamento

Si vuole implementare un gestore per un pool di N risorse equivalenti (intercambiabili).

- `int richiesta()`: restituisce l'indice della risorsa attribuita. È bloccante se non ci sono risorse libere.
- `void rilascio(int k)`: rilascia la risorsa k-esima (acquisita in precedenza).

23.14.2. Codice: Classe Gestore

```
public class Gestore {
    // true = risorsa occupata, false = risorsa libera
    private boolean[] risOccupate;
    private int numOccupate;

    // n: numero di risorse da gestire
    public Gestore(int n) {
        risOccupate = new boolean[n];
        // (i valori default 'false' indicano che sono tutte libere)
    }

    // Mutua esclusione necessaria per evitare di attribuire
    // la stessa risorsa a più utilizzatori
    public synchronized int richiesta() throws InterruptedException {
        // Se non ci sono risorse libere (tutte occupate)
        while (numOccupate == risOccupate.length) {
            // mi blocco
            wait();
        }

        // Trovo il primo elemento che vale 'false' (libero)
        int i = 0;
        for (; i < risOccupate.length && risOccupate[i]; i++);

        // Marco la risorsa come occupata
        risOccupate[i] = true;
        numOccupate++;
        return i;
    }

    public synchronized void rilascio(int k) {
        // Marco la risorsa come libera
        risOccupate[k] = false;
        numOccupate--;
    }
}
```

```

        // Uso notify() perché le condizioni di ottimizzazione sono rispettate:
        // 1. Tutti attendono la stessa condizione (numOccupate < length)
        // 2. Solo un thread può beneficiare della risorsa liberata.
        notify();
    }
}

```

23.14.3. Codice: Classe Utilizzatore

```

public class Utilizzatore extends Thread {
    private Gestore g;
    private int numRichieste;
    private static final int DEFAULT_NUM_RICHIESTE = 100;

    public Utilizzatore(String n, Gestore g, int r) {
        super(n);
        this.g = g;
        this.numRichieste = r;
    }

    public Utilizzatore(String n, Gestore g) {
        // Chiama l'altro costruttore
        this(n, g, DEFAULT_NUM_RICHIESTE);
    }

    public void run() {
        try {
            for (int i = 0; i < numRichieste; i++) {
                // 1. Chiede la risorsa (bloccandosi se necessario)
                int k = g.richiesta();
                System.out.println(getName() + ": ho ottenuto la risorsa " + k);

                // 2. Simula utilizzo della risorsa
                sleep((long) (Math.random() * 1000));

                // 3. Rilascia la risorsa
                g.rilascio(k);
                System.out.println(getName()
                    + ": ho rilasciato la risorsa " + k);
            }
        } catch (InterruptedException ie) {
            return;
        }
    }
}

```

```
}  
}
```

23.14.4. Codice: Classe Prova (Main)

```
import java.util.Scanner;  
  
public class Prova {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Quanti utilizzatori?");  
        int u = sc.nextInt();  
        System.out.println("Quante risorse?");  
        int r = sc.nextInt();  
  
        Gestore g = new Gestore(r);  
  
        // Avvia tutti i thread utilizzatori  
        for (int i = 0; i < u; i++)  
            new Utilizzatore("U" + i, g).start();  
    }  
}
```

23.15. Il Metodo join()

23.15.1. Scopo e Funzionamento

Supponiamo che un thread t1 voglia attendere la fine dell'esecuzione di un thread t2.

- Per farlo, t1 deve chiamare il metodo join() sull'oggetto t2 (cioè t2.join();).
- Il thread chiamante (t1) si blocca dentro il metodo join().
- t1 esce dalla join() (si sblocca) quando il thread t2 arriva in fondo al suo metodo run() (termina).
- Se t2 ha già terminato quando join() viene chiamato, la chiamata non ha effetto (non è bloccante).

23.15.2. Versioni del Metodo

Esistono tre versioni di join():

- `public final void join(long millis) throws InterruptedException` — blocca il chiamante fino a quando il thread termina, o fino a quando sono trascorsi millis millisecondi.
- `public final void join(long millis, int nano) throws InterruptedException` — come sopra, ma con precisione al nanosecondo.
- `public final void join() throws InterruptedException` — attende che il thread finisca la propria esecuzione (timeout infinito). Equivale a `join(0)`.

23.15.3. Esempio: Analizzatore (Numero Primo)

Il thread main attiva un thread ausiliario (Analizzatore) per verificare se un numero è primo. Il main attende che l'ausiliario termini per ottenere il risultato.

Classe Lancia (Main):

```
public class Lancia {
    public static void main(String[] args) {
        // Converte l'argomento da riga di comando in int
        int n = Integer.parseInt(args[0]);
        Analizzatore a = new Analizzatore(n);
        a.start();

        try {
            // Si blocca, fino a quando 'a' non arriva in fondo al run()
            a.join();

            // Recupero il risultato DOPO che 'a' ha finito
            boolean r = a.getRisultato();
            System.out.println(r ? "primo" : "non primo");
        } catch (InterruptedException ie) {
            // nothing
        }
    }
}
```

```
    }  
  }  
}
```

Classe Analizzatore (Thread):

```
public class Analizzatore extends Thread {  
    private int n;  
    private boolean risultato;  
  
    public Analizzatore(int n) {  
        this.n = n;  
    }  
  
    public void run() {  
        risultato = primo(n);  
    }  
  
    // Metodo per recuperare il risultato  
    public boolean getRisultato() {  
        return risultato;  
    }  
  
    private boolean primo(int n) {  
        for (int i = 2; i < n; i++) {  
            if (n % i == 0)  
                return false;  
        }  
        return true;  
    }  
}
```

23.16. Interrompere un Thread (interrupt())

23.16.1. Interruzione Cooperativa

L'interruzione delle operazioni in Java deve essere cooperativa. Questo significa che il thread deve controllare attivamente se gli è stato chiesto di arrestare le proprie operazioni.

23.16.2. Il Meccanismo interrupt()

- Ad ogni thread è associato un interrupt status (un flag).
- Per inviare un "interrupt" a un thread, si deve chiamare il suo metodo `public void interrupt()`.

23.16.3. Scenari di Interruzione

Quando si chiama `interrupt()` su un thread, possono succedere due cose:

1. Se il thread è bloccato su un metodo come `sleep()`, `wait()` o `join()`, il thread esce immediatamente dal blocco lanciando una `InterruptedException`. Importante: in questo caso, l'interrupt status NON viene settato (o, se lo era, viene resettato).
2. Se il thread sta eseguendo codice "normale" (non è bloccato), viene semplicemente settato l'interrupt status (il flag). Il thread continua la sua esecuzione.

23.16.4. Controllare lo Stato di Interruzione

Un thread può controllare il proprio interrupt status in due modi:

- `public static boolean interrupted()`
 - Metodo statico. Controlla lo status del thread corrente.
 - Restituisce `true` se il flag è settato, `false` altrimenti.
 - Importante: resetta l'interrupt status (rimette il flag a `false`) dopo averlo letto.
- `public boolean isInterrupted()`
 - Metodo d'istanza.
 - Restituisce `true` se il flag è settato, `false` altrimenti.
 - Non resetta lo status.

23.16.5. Esempio: Creatore ed Esecutore

Il main (Creatore) attiva un thread Esecutore. Dopo 2 secondi, il main interrompe l'esecutore. L'esecutore, quando interrotto, termina.

Classe Creatore (Main):

```
public class Creatore {
    public static void main(String[] args) {
        Esecutore e = new Esecutore();
        e.start();

        try {
            // Dorme per 2 secondi
            Thread.sleep(2000);
        } catch (InterruptedException ie) {
            // do nothing
        }

        // Invia l'interrupt all'esecutore
        e.interrupt();
    }
}
```

Classe Esecutore (Thread):

```
public class Esecutore extends Thread {
    public void run() {
        // Il thread controlla il proprio interrupt status
        // Usa interrupted() che legge e resetta il flag
        while (!interrupted()) {
            // Continua a fare il suo lavoro
            fai();
        }
        // Il loop termina quando l'interrupt() viene chiamato
        // e il check !interrupted() restituisce 'false'.
    }

    private void fai() {
        System.out.print(".");
    }
}
```

24. Classi Wrapper

24.1. Definizione e Scopo

Le **classi wrapper** sono classi che corrispondono ai tipi primitivi.

Tipo Primitivo	Classe Wrapper
boolean	Boolean
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character

Sono utili principalmente per due motivi:

1. Rappresentare valori primitivi come oggetti: necessario in contesti che richiedono tipi riferimento (oggetti), come le collezioni (ad esempio, una Map). Se la chiave è un valore intero, si può passare un Integer ma non un int.
2. Metodi di utilità: raccolgono metodi di utilità relativi al tipo primitivo corrispondente.

24.2. Creazione di Istanze (Costruttori e valueOf)

Gli oggetti wrapper sono **immutabili**: una volta creato, il valore interno non può essere modificato. Il riferimento può cambiare puntando a un nuovo oggetto, ma l'oggetto originale resta invariato.

Si possono creare in diversi modi:

- Costruttore (deprecato):

```
// Crea un oggetto a partire da un int
Integer i1 = new Integer(10);
// Crea un oggetto a partire da una stringa
Integer i2 = new Integer("30");
```

- Metodo statico valueOf() (preferito):

Questo metodo può restituire un riferimento a un oggetto creato in precedenza con lo stesso valore (caching), ottimizzando l'uso della memoria.

```
Integer i4 = Integer.valueOf(50);  
Integer i5 = Integer.valueOf("60");
```

24.3. Costanti e Metodi di Utilità

Le classi wrapper forniscono metodi per convertire il valore e costanti utili.

- Ottenere il valore primitivo:

```
Integer i6 = new Integer(70);  
// Restituisce il valore dell'Integer come un int  
int x = i6.intValue(); // x vale 70
```

- Costanti:

```
Integer.MAX_VALUE; // (2^31 - 1)  
Integer.MIN_VALUE; // (-2^31)
```

Nota: tra i metodi di utilità più comuni vi è `parseInt()`, che converte una stringa in un tipo `int` primitivo (es. `int n = Integer.parseInt("123");`). Questo è diverso da `Integer.valueOf("123")` che restituisce un oggetto `Integer`.

25. Autoboxing e Unboxing

Si intende la conversione automatica da tipo primitivo a classe wrapper (autoboxing) e viceversa (unboxing).

25.1. Autoboxing (Da Primitivo a Wrapper)

L'autoboxing avviene automaticamente quando un valore primitivo viene assegnato a un riferimento wrapper o passato come parametro a un metodo che richiede un wrapper.

```
// Assegnamento diretto  
Integer i7 = 80;  
// Il compilatore lo traduce automaticamente in:  
Integer i7 = Integer.valueOf(80);
```

Esempio con metodi:

```
void met(Integer x) {  
    // ...  
}  
  
// Chiamata  
int z = 10;  
met(z); // Autoboxing: z (int) viene convertito in Integer
```

25.2. Unboxing (Da Wrapper a Primitivo)

L'unboxing avviene automaticamente quando un oggetto wrapper viene assegnato a una variabile primitiva.

```
// Autoboxing per creare l'oggetto  
Integer J = 23;  
  
// Unboxing  
int w = J;  
  
// Il compilatore lo traduce automaticamente in:  
int w = J.intValue();
```

26. Java Memory Model (JMM)

26.1. Il Problema della Visibilità (Esempio)

Consideriamo il seguente codice, dove un thread main cerca di fermare un altro thread (Altro) impostando una variabile booleana condivisa fine.

Classe Main (Thread 1):

```
class Main {
    public static void main(String[] args) {
        System.out.println("Faccio partire un altro thread");
        Altro t1 = new Altro();
        t1.start();

        Scanner sc = new Scanner(System.in);
        String l = sc.next(); // Attende input

        System.out.println("Metto fine a true");
        t1.fine = true;

        System.out.println("Leggo ancora da tastiera");
        l = sc.next();
    }
}
```

Classe Altro (Thread 2):

```
public class Altro extends Thread {
    boolean fine = false;

    public void run() {
        while (!fine) {
            // Busy Loop: ciclo che consuma CPU
            for (int i = 0; i < 1000000; i++);
        }
        System.out.print("ESCO");
    }
}
```

Problema: in assenza di sincronizzazione, non c'è garanzia che il thread Altro legga il valore più recente della variabile fine (scritto dal thread Main).

Per ottimizzazione, il compilatore (JIT) o il processore potrebbero memorizzare (mettere in cache o in un registro) il valore false di fine all'inizio del loop. Vedendo che il loop stesso non modifica fine, il

compilatore potrebbe sostituire `while(!fine)` con `while(true)`, causando un ciclo infinito. L'output "ESCO" non verrebbe mai stampato.

26.2. Atomicità vs. Visibilità

- Atomicità: il JMM garantisce che le operazioni di lettura/scrittura su variabili (eccetto `double` e `long`) siano atomiche. Se T1 esegue `x=5` e T2 esegue `x=-100`, il valore finale sarà o 5 o -100, non un mix dei due. Questo previene l'interferenza.
- Visibilità: l'atomicità da sola non assicura la visibilità, ovvero che una modifica fatta da un thread sia vista dagli altri.

Il Java Memory Model (JMM) è l'insieme di regole che stabiliscono l'ordinamento degli accessi alla memoria e quando le modifiche sono visibili in modo garantito.

26.3. La Relazione "Happens-Before"

Per garantire la visibilità e restringere l'insieme dei valori possibili all'ultimo valore scritto, il JMM definisce la relazione **"happens-before"** ("avviene prima"). Se l'azione A *happens-before* l'azione B, allora gli effetti di A sono visibili a B.

Principali regole *happens-before*:

- Il rilascio di un lock (fine blocco `synchronized` o `unlock()`) *happens-before* ogni successiva acquisizione dello stesso lock.
- La scrittura su una variabile `volatile` *happens-before* ogni successiva lettura della stessa variabile `volatile`.
- La chiamata a `thread.start()` *happens-before* qualsiasi azione nel thread che è stato fatto partire.
- Le operazioni in un thread *happens-before* il ritorno di una `join()` su tale thread.

Soluzione all'esempio: per risolvere il problema della visibilità dell'esempio precedente, la soluzione più semplice è dichiarare la variabile `fine` come `volatile`:

```
public volatile boolean fine = false;
```

Questo crea una relazione *happens-before* tra la scrittura `t1.fine = true` (in Main) e la lettura `while(!fine)` (in Altro), garantendo che il loop termini.

Nota: `volatile` garantisce visibilità, ma non atomicità per operazioni composte (come `i++`, che è lettura-modifica-scrittura). Per quelle, serve `synchronized` o classi `java.util.concurrent.atomic`.

26.4. Riordinamento delle Istruzioni (Reordering)

In assenza di sincronizzazione, un thread può vedere le operazioni fatte da un altro thread "al contrario" (riordinate).

Esempio:

- Stato iniziale: $A == B == 0$

Thread 1	Thread 2
1) $r2 = A;$	3) $r1 = B;$
2) $B = 1;$	4) $A = 2;$

Risultato "impossibile": è possibile ottenere $r2 == 2$ e $r1 == 1$?

Spiegazione: sì. Ai compilatori (JIT) e ai processori è permesso riordinare le istruzioni di un thread, a patto che il riordinamento non influisca sul comportamento del singolo thread.

Scenario riorinato:

1. Thread 1 riordina 2) e 1) (lecito, non cambia la sua logica interna).
2. Thread 2 riordina 4) e 3) (lecito).
3. Esecuzione:
 - T1 esegue $B = 1;$
 - T2 esegue $A = 2;$
 - T1 esegue $r2 = A;$ (legge 2)
 - T2 esegue $r1 = B;$ (legge 1)

Questo malfunzionamento è dovuto a incorretta sincronizzazione.